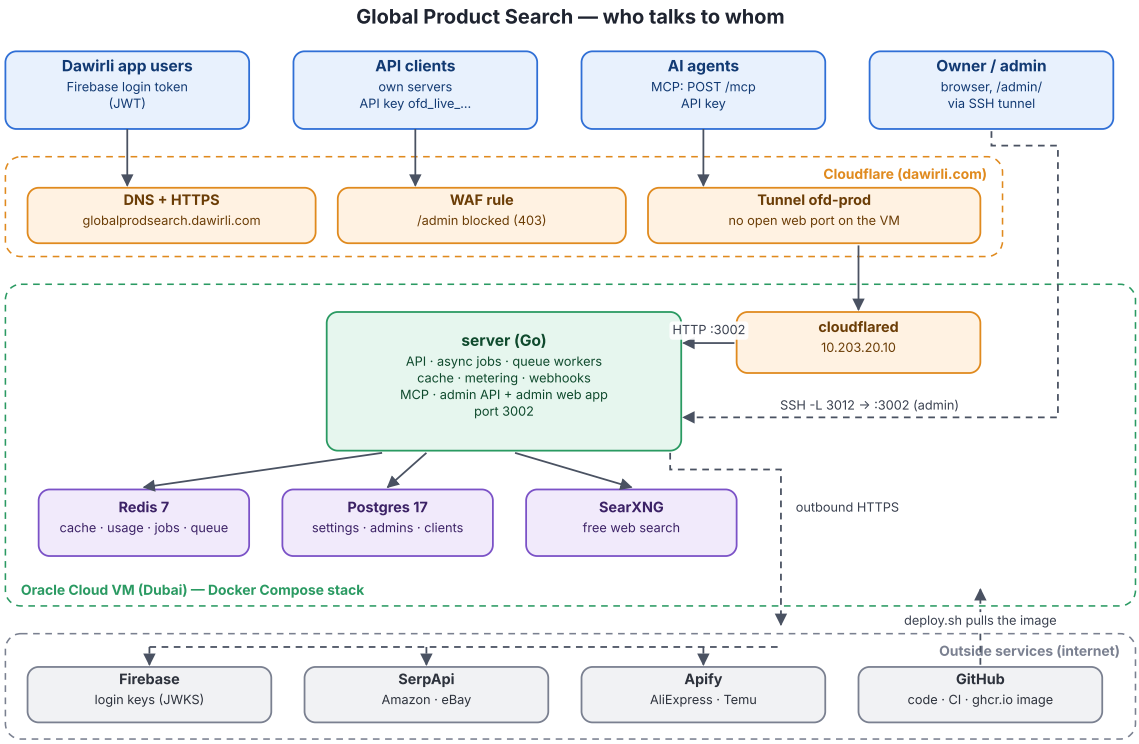
Global Product Search

Full Service Reference

Owis Find Deal Engine · design, code, deployment and operations



Edition: 11 November 2026 — state after step 14b parts 3 + 4 (deployed and tested)
Service: https://globalprodsearch.dawirli.com · **Code:** github.com/owisdev/claude-ai-clould-repo → Owis_Find_Deal_Engine/server
Production: Oracle Cloud VM (Dubai) · Docker Compose · Cloudflare Tunnel
For: the service owner

Owis Find Deal Engine · the product search service behind the Dawirli app

Edition	11 November 2026 — state after step 14b parts 3 + 4 (PR #26, deployed and tested)
Public address	`https://globalprodsearch.dawirli.com`
Repository	`github.com/owisdev/claude-ai-clould-repo`, folder `Owis_Find_Deal_Engine/`
Server code	`Owis_Find_Deal_Engine/server/` — Go 1.26, module `owis_find_deal_engine`
Production	Oracle Cloud VM (Dubai), Docker Compose, Cloudflare Tunnel
Owner	the service owner (you) — this document is written for you

This is the one document that describes the whole service as it is today: what it does, how it is built, where every part lives in the Go code, how it runs in the cloud, how you operate and test it, and what is still to do. Other documents go deeper on single topics: the server `README.md` (developer reference), `PLAN.md` (the plan and every decision), `STATUS.md` (the log of every step and test), the developer page `/docs/` and the OpenAPI file `/api/v1/openapi.json` (for API clients).

> **How to read it.** Parts 1–3 give the big picture. Parts 4–12 explain each mechanism with its code. Parts 13–17 are the reference you look things up in (API, admin, settings, data). Parts 18–21 are the hands-on handbook: every command for your PC and the VM, tests, troubleshooting. Part 22 is what is left.

Contents

1. What the service does
2. Architecture at a glance
3. Production deployment in the cloud
4. From code change to production (CI/CD)
5. Inside the server: code structure
6. Life of the server process
7. The request pipeline: identity, limits, allowance
8. The search engine: countries, shops, sources
9. The cache
10. The queue
11. Waiting search vs asynchronous search
12. Webhooks
13. Restarts, failures and how the service recovers
14. AI agents (MCP)
15. Public API reference
16. Admin panel and admin API
17. Settings reference
18. Data stores: Postgres and Redis
19. Usage, cost and budget
20. Command handbook: PC and VM
21. Testing
22. Troubleshooting
23. Current status and remaining work
24. Glossary

1. What the service does

A user types a product ("usb c hub") and picks a country (`usa` , `ksa` , `jor`). The service searches the online shops that deliver to that country and returns **one merged list of offers**: title, price, currency, image, link to the shop's product page — plus the status of every shop.

Who calls it

Caller	How it identifies itself	Typical use
Dawirli app users	Firebase login token (JWT)	search from the app; asynchronous search with polling
API clients (own servers, partners)	API key `ofd_live_…` made in the admin panel	search, asynchronous search, signed webhooks
AI agents (Claude, other MCP clients)	the same API keys, on `POST /mcp`	tools `search_products` , `get_search_results` , `list_countries`
The owner (admin)	Firebase login + admin list, through an SSH tunnel	settings, API clients, usage and cost, test searches

Countries and shops today (from `markets.json`):

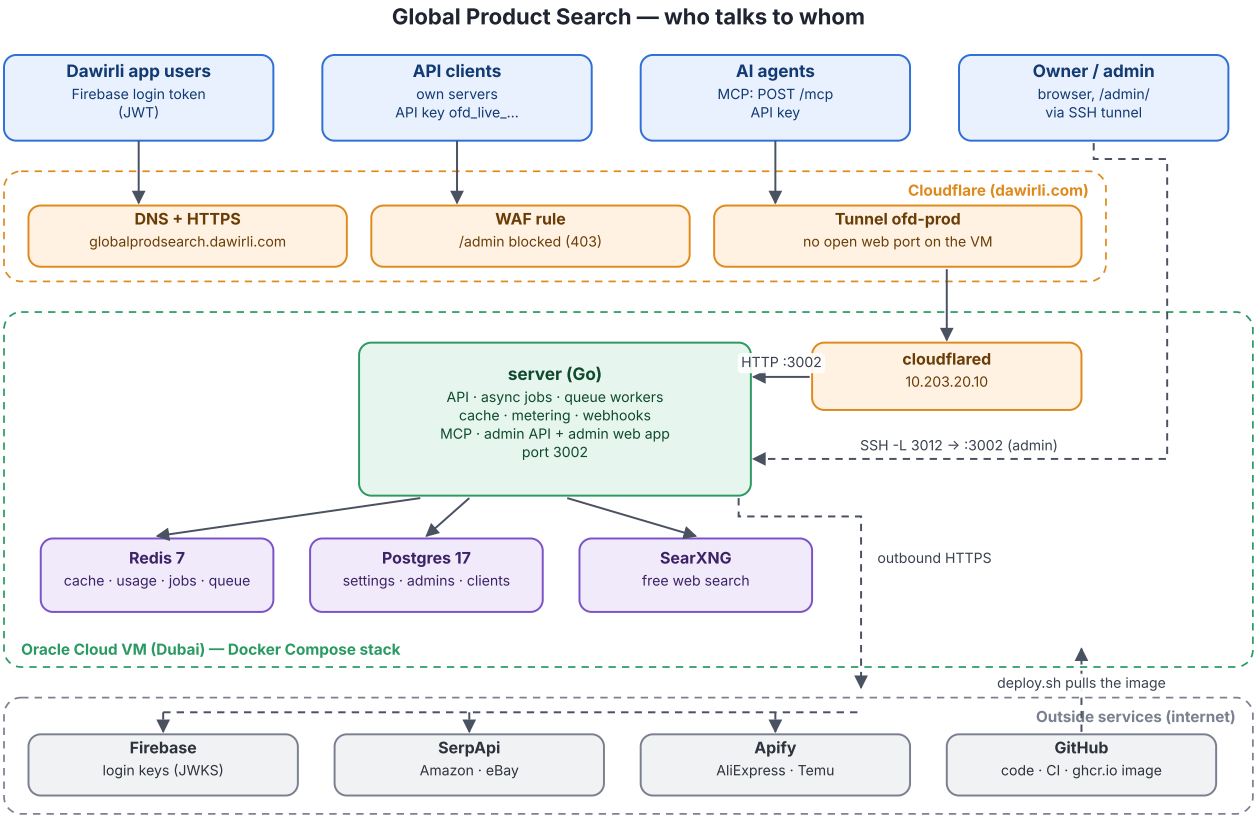
Country	Code	Currency	Shops (site)
United States	`usa`	USD	Amazon (amazon.com), eBay, AliExpress, Temu, SHEIN (us.shein.com)
Saudi Arabia	`ksa`	SAR	Amazon (amazon.sa), eBay, AliExpress, Temu, SHEIN (ar.shein.com)
Jordan	`jor`	JOD (shops show USD)	Amazon (amazon.com), AliExpress, Temu, SHEIN (ar.shein.com)

How each shop is searched in production (the "rules per shop", step 14b):

Shop	Source	Time limit	Group	Typical time (Nov 2026)
Amazon	SerpApi Amazon engine	15 s	fast	2–4 s
eBay	SerpApi eBay engine	15 s	fast	3–4 s
AliExpress	Apify scraper	90 s	slow	8 s – 85 s (varies a lot)
Temu	Apify scraper	45 s	slow	7 – 25 s
SHEIN	**off** (no working source yet)	—	—	—

Key ideas in one paragraph. Each shop is searched on its own, in parallel, through a **queue** with a cap on how many paid runs go at once. Results are **cached per shop for 48 hours**, so a repeated search is instant and free, and countries share a shop's results when the request is the same. A shop that fails shows its **last known results** (up to 7 days old, dated). The app and agents use the **asynchronous search**: they get the fast shops within seconds and the slow ones fill in later; API clients can also get **signed webhooks**. Every live search counts **once** against a daily allowance; cached answers never count.

2. Architecture at a glance



The parts

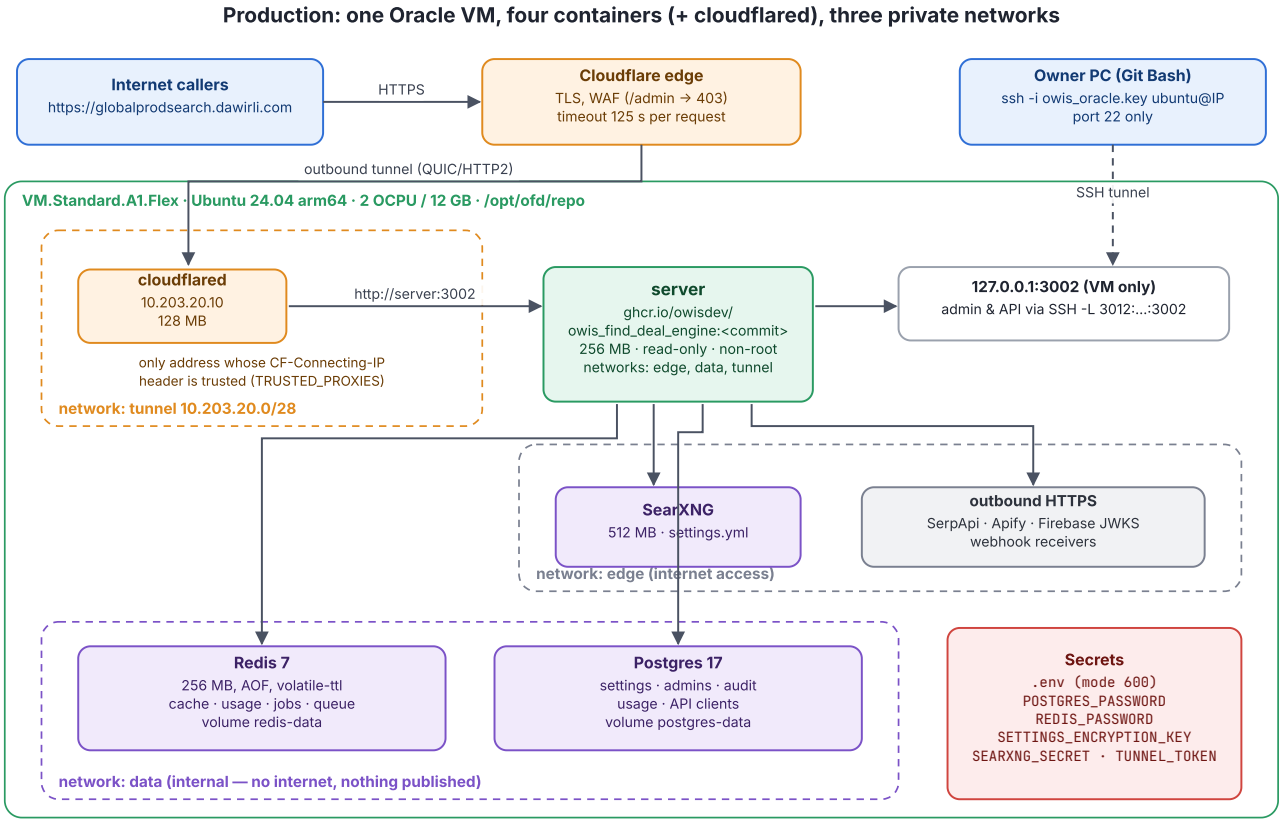
Part	What it is	Where
Cloudflare	DNS for `dawirli.com`, HTTPS certificate, WAF rule that blocks `/admin` from the internet, and the **Tunnel** `ofd-prod` that carries traffic to the VM without any open web port	Cloudflare dashboard
cloudflared	the tunnel's connector, a container next to the server; it opens an outbound connection to Cloudflare	`compose.prod.yml`
server	one Go program: public API, asynchronous jobs, queue workers, cache front, metering, webhooks, MCP endpoint, admin API and the embedded admin web app	`server/cmd/owis_find_deal_engine` + `server/internal/*`
Redis 7	fast shared state: search cache, usage counters, search jobs, the queue	`docker-compose.yml`
Postgres 17	durable state: runtime settings (+ history), admins, audit log, API clients and keys, daily usage	`docker-compose.yml`, migrations in `internal/store/migrations`
SearXNG	free meta search engine, used only by shops whose source is `web` (none in production today)	`docker-compose.yml`, `deploy/searxng/settings.yml`
Firebase	signs users' and admins' login tokens; the server checks them with Google's public keys (JWKS)	Firebase console
SerpApi	paid search API: Amazon Search and eBay Search engines (also Google Shopping / web for `web` shops)	serpapi.com
Apify	paid scrapers from the Apify store for AliExpress and Temu	console.apify.com
GitHub	code, pull requests, CI (GitHub Actions), and the container registry `ghcr.io`	github.com

Design principles that shaped the code:

- **One binary, one deployment.** The admin web app is compiled into the Go binary; there is no separate frontend server.

- **Settings live in the database** and change without a restart (the server rebuilds its search setup and swaps it in).
- **Nothing is exposed but the tunnel.** Only SSH (22) is open on the VM; Redis and Postgres are on an internal Docker network with no internet.
- **Pay once.** Cache per shop, joining identical searches, counting a search only when something new was fetched.
- **Degrade, don't fail.** A shop that fails shows its last known results; only when every shop fails does a search fail.

3. Production deployment in the cloud



3.1 The VM

Provider	Oracle Cloud **Always Free**, home region **UAE East (Dubai)**
Shape	`VM.Standard.A1.Flex` (Ampere, **arm64**), 2 OCPUs, 12 GB RAM, ~47 GB boot volume
OS	Canonical Ubuntu 24.04
Network	own VCN `owis-vcn`, public subnet, public IPv4 (see Oracle console → Compute → Instances)
Login	`ssh -i ~/.ssh/owis_oracle.key ubuntu@<public IP>` — key only on your PC
Open ports	**22 only** (SSH, key login only, no root, no passwords)
Repository on the VM	`/opt/ofd/repo` (a clone of `main`); server folder `/opt/ofd/repo/Owis_Find_Deal_Engine/server`
Secrets	`/opt/ofd/repo/Owis_Find_Deal_Engine/server/.env`, mode 600, never in Git
Deploy log	`/opt/ofd/deploys.log`
Budget guard	Oracle budget `free-tier-guard` (€1/month, email on any real charge)

The VM was prepared once by `deploy/bootstrap-vm.sh`: system updates, Docker Engine + Compose, automatic security updates, SSH hardening (keys only), Docker log rotation, clone of the repository to `/opt/ofd/repo`, `.env` created from `.env.example` with mode 600, rpcbind turned off. The script is safe to run again.

3.2 The containers

The stack is `docker-compose.yml` plus `compose.prod.yml` (production image from `ghcr.io` and the `cloudflared` container). Every container is **read-only**, runs as **non-root**, drops **all Linux capabilities**, has `no-new-privileges`, a memory and process limit, a health check and rotated logs.

Container	Image	Memory	Networks	Notes
`owis_find_deal_engine-server-1`	`ghcr.io/owisdev/owis_find_deal_engine:<commit>`	256 MB	edge, data, tunnel	distroless image (~19 MB); health check `/owis_find_deal_engine healthcheck`; publishes `127.0.0.1:3002` only
`owis_find_deal_engine-cloudflared-1`	`cloudflare/cloudflared:2026.10.7`	128 MB	tunnel (`10.203.20.10`)	started only when `TUNNEL_TOKEN` is in `.env` (profile `tunnel`)
`owis_find_deal_engine-redis-1`	`redis:7-alpine`	320 MB (Redis max 256 MB)	data	password, AOF persistence, `maxmemory-policy volatile-ttl`, volume `redis-data`
`owis_find_deal_engine-postgres-1`	`postgres:17-alpine`	256 MB	data	volume `postgres-data`
`owis_find_deal_engine-searxng-1`	`searxng/searxng:2026.11.2-…` (pinned)	512 MB	edge	settings from `deploy/searxng/settings.yml`

3.3 The three Docker networks

Network	Members	Internet	Why
`edge`	server, SearXNG	yes	the server calls SerpApi, Apify, Firebase JWKS and webhook receivers; SearXNG queries search engines
`data`	server, Redis, Postgres	**no** (internal)	the databases can never be reached from outside, nor reach out
`tunnel`	server, cloudflared	—	subnet `10.203.20.0/28`; cloudflared has the **fixed address** `10.203.20.10`, the only address whose `CF-Connecting-IP` header the server believes (`TRUSTED_PROXIES`)

3.4 How a request reaches the server

1. A caller opens `https://globalprodsearch.dawirli.com/api/v1/...`.
2. Cloudflare terminates HTTPS. The **WAF rule** answers `403` for any path starting with `/admin` (the admin panel is not public until Cloudflare Access, step 6). Bot Fight Mode is **off** so apps and agents get through.
3. Cloudflare sends the request through the **tunnel**: `cloudflared` on the VM keeps an outbound connection to Cloudflare, so no inbound port is needed. In the Cloudflare dashboard the tunnel's public hostname points to `http://server:3002`.
4. cloudflared forwards to `server:3002` on the `tunnel` network and adds `CF-Connecting-IP` (the caller's real address).
5. The server's outermost handler `api.WithClientIP` takes the caller's address from that header only when the connection comes from `10.203.20.10`; otherwise it uses the connection address.

Cloudflare's limit: Cloudflare waits at most **125 s** for an answer (then `524`). The waiting search ends by 92 s in production (slowest shop 90 s + 2 s), and the asynchronous search answers at once, so this limit is not reached.

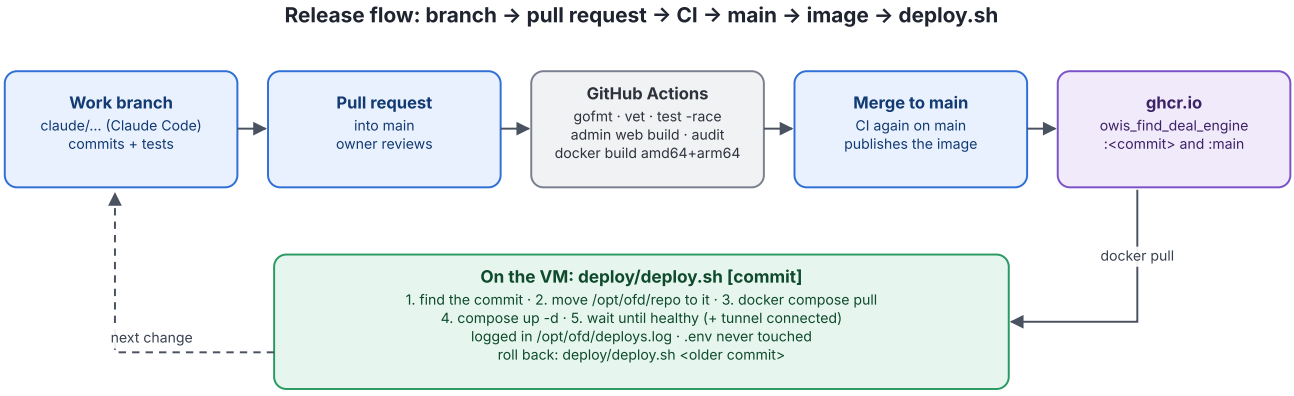
3.5 Domains, DNS, email

- Domain **`dawirli.com`** (Cloudflare Registrar). The Dawirli app uses `dawirli.com` / `www.dawirli.com`; this service is **`globalprodsearch.dawirli.com`** (API, `/docs`, `/mcp`; `/admin` blocked).
- Cloudflare **Email Routing**: `admin@dawirli.com` → the owner's Gmail (for Firebase and, later, Cloudflare Access mails).
- When the Dawirli app calls the API from a **browser**, set `CORS_ALLOWED_ORIGINS=https://dawirli.com,https://www.dawirli.com` (mobile apps don't need CORS).

3.6 Secrets: where each one lives

Secret	Where	Notes
`POSTGRES_PASSWORD`, `REDIS_PASSWORD`, `SEARXNG_SECRET`	VM `.env`	random 64-hex (`openssl rand -hex 32`)
`SETTINGS_ENCRYPTION_KEY`	VM `.env` + **an offline copy** (password manager)	encrypts the secret settings in Postgres (AES-256-GCM) and derives webhook secrets; losing it means setting the SerpApi key / Apify token again and making new webhook secrets
`TUNNEL_TOKEN`	VM `.env`	connects cloudflared to tunnel `ofd-prod`
`SERPAPI_KEY`, `APIFY_TOKEN`	Postgres `settings` table, **encrypted**, write-only	set in the admin panel (Settings) or with the settings CLI
API keys `ofd_live_…`	only their **SHA-256 hash** in Postgres	shown once when made
Webhook secrets `whsec_…`	**not stored at all**: derived from `SETTINGS_ENCRYPTION_KEY` + a random salt in Postgres	shown once when made
SSH key `owis_oracle.key`	your PC only	

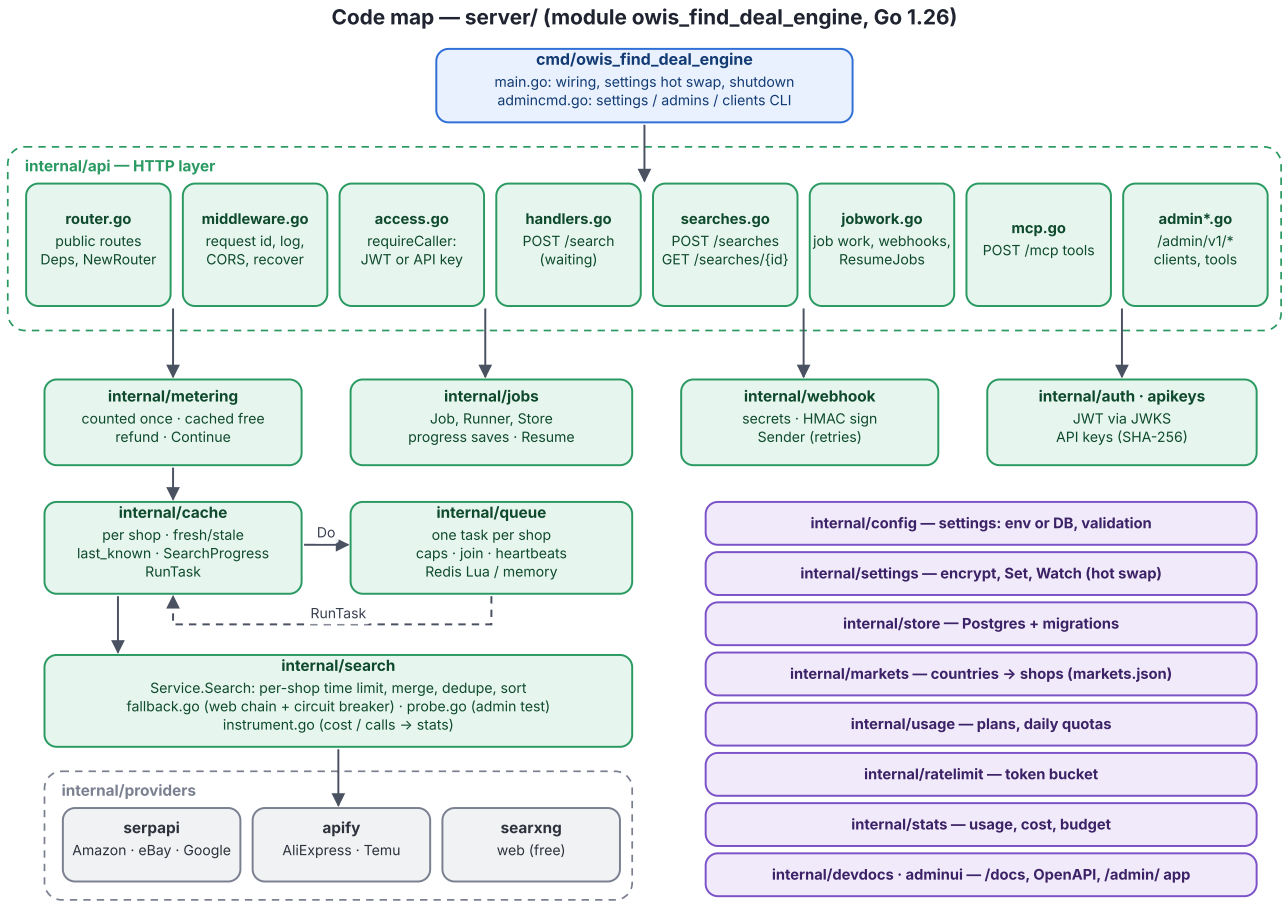
4. From code change to production (CI/CD)



1. **Work branch.** Changes are made on a branch (Claude Code sessions use `claude/<name>`) with tests, then pushed.
2. **Pull request into `main`.** You review and merge on GitHub. Each step of the plan is one PR.
3. **CI** (`.github/workflows/server.yml`), on every push or PR touching `Owis_Find_Deal_Engine/server/**`: - job **test**: `gofmt`, `go mod tidy` check, `go vet`, `go test -race` (with a throwaway Postgres for the store tests), `govulncheck`; - job **admin-web**: `npm ci`, typecheck + build, `npm audit`; - job **docker**: validates the compose files and scripts, builds the image, smoke-tests it (it must refuse to start without configuration), then builds **amd64 + arm64**. On `main` only, it **publishes** `ghcr.io/owisdev/owis_find_deal_engine:<commit>` and `:main` (label `org.opencontainers.image.revision=<commit>`).
4. **Deploy by hand** (decision Q1: no automatic deploy). On the VM: `deploy/deploy.sh` (newest `:main`) or `deploy/deploy.sh <commit>`. `deploy.sh` does five steps: find the commit (only commits of `main`), move `/opt/ofd/repo` to that commit (so compose files match the image), pull images, `compose up -d`, wait until the server is healthy and the tunnel connected; it prints the version, logs the deploy to `/opt/ofd/deploys.log`, and on failure shows the log and the **rollback** command (`deploy/deploy.sh <previous>`). It never touches `.env`.
5. **Migrations** run automatically at server start (`store.Migrate`); rolling back past a migration is not supported (restore a backup instead).

The Docker image (`Dockerfile`) is built in three stages: Node builds the admin web app → Go cross-compiles a static binary (no cgo) with the web app embedded → `distroless/static` as non-root. The build runs on the CI machine's own platform and cross-compiles for arm64, so no emulation is needed.

5. Inside the server: code structure



All paths below are under `Owis_Find_Deal_Engine/server/`.

5.1 Packages

Package	Lines*	Responsibility	Key types / functions
`cmd/owis_find_deal_engine`	1250	the program: loads settings, wires everything, swaps the search setup on settings changes, graceful shutdown; admin command line	`run`, `buildApp`, `shared`, `app`, `queueOptions`, `adminCommand`
`internal/api`	3340	HTTP layer: routes, middleware, auth check, public handlers, asynchronous search, job work and webhooks, MCP, admin API	`NewRouter`, `Deps`, `NewAdminRouter`, `WithClientIP`, `ResumeJobs`
`internal/metering`	260	what a search costs the caller: cached free, live counted once, refunds	`Service.Search`, `SearchProgress`, `Check`, `Continue`
`internal/cache`	775	per-shop cache, freshness, last known results, live fetch through the queue, answer so far	`Cache.Lookup`, `Search`, `SearchProgress`, `RunTask`, `Forget`
`internal/queue`	820	one task per shop and title, worker pool, caps, joining, restarts (Redis Lua or memory)	`Queue.Do`, `Enqueue`, `Wait`, `SetRunner`, `SetOptions`, `Close`, `ClassOf`
`internal/jobs`	585	asynchronous search jobs: store (Redis / memory), runner, progress saves, resume	`Job`, `Runner.Start`, `Resume`, `Close`, `Webhook.Wants`
`internal/search`	1210	the search service: per-shop time limits, provider calls, merge, dedupe, sort, currency; web provider chain with circuit breaker; admin probe	`Service.Search`, `NewFallback`, `Probe`, `MergeShops`, `NormalizeCurrency`, `Relevant`
`internal/providers/serpapi`	755	SerpApi: Amazon, eBay, Google Shopping / web	`NewAmazon`, `NewEbay`, `New`

Package	Lines*	Responsibility	Key types / functions
`internal/providers/apify`	400	Apify store scrapers, one per shop	`New`, `Client.Search` (run-sync-get-dataset-items)
`internal/providers/searxng`	315	SearXNG web search	`New`
`internal/markets`	415	countries → shops catalog, product-link rules	`Load`, `Catalog.Country`, `WithProviders`, `Target.ProductLink`
`internal/auth`	345	JWT verification with a cached JWKS	`NewJWKS`, `NewVerifier`, `Verify`, `UserFrom`
`internal/apikeys`	380	API keys: format, SHA-256, cached lookup, scopes, countries, IP allow-list	`Generate`, `Authenticator.Authenticate`, `HasScope`, `AllowedCountry`, `AllowedIP`
`internal/usage`	305	plans and daily allowances (Redis / memory counters)	`ParsePlans`, `Quota.Take`, `Refund`, `Status`
`internal/ratelimit`	95	token bucket per caller	`New`, `Allow`
`internal/webhook`	360	webhook secrets, HMAC signing and verifying, URL checks, sender with retries	`NewSecrets`, `Sign`, `Verify`, `CheckURL`, `Sender.Deliver`
`internal/config`	725	all settings: parse, defaults, validation; bootstrap vs runtime keys	`Load`, `LoadFrom`, `Config`, `ShopRule`, `Runtime`
`internal/settings`	300	settings in Postgres: encryption, validated changes, import from `.env`, change watcher	`Manager.Set`, `Config`, `ImportEnv`, `Watch`, `Cipher`
`internal/store`	910	Postgres access and migrations	`Open`, `Migrate`, settings / admins / audit / clients / usage functions
`internal/stats`	585	usage counters, dashboard report, monthly budget alert	`Recorder`, `BuildReport`, `CheckBudget`
`internal/devdocs`	60	serves `/docs/` and `/api/v1/openapi.json` (static files)	`Page`, `OpenAPI`
`internal/adminui`	100	serves the embedded admin web app at `/admin/`	`Handler`
`web/admin`	—	admin web app source (React + TypeScript + Vite)	`src/pages/*.tsx`

* without tests, November 2026.

5.2 Folder layout

```
Owis_Find_Deal_Engine/
├── PLAN.md                plan, decisions, steps
├── STATUS.md              log of every step, tests, handoff
├── docs/                  guides (PDF) and their generators, this reference
└── server/
    ├── cmd/owis_find_deal_engine/   main.go (wiring), admincmd.go (CLI)
    ├── internal/
    │   ├── api/           router.go, middleware.go, clientip.go, access.go,
    │   │                  handlers.go, searches.go, jobwork.go, mcp.go,
    │   │                  admin.go, admin_handlers.go, admin_clients.go, respond.go
    │   ├── cache/  queue/  jobs/  metering/  search/  providers/{serpapi,apify,searxng}
    │   ├── markets/ (markets.json)   auth/ (+authtest)   apikeys/   usage/   ratelimit/
    │   ├── webhook/  config/  settings/  store/ (migrations/*.sql)  stats/
    │   └── devdocs/ (static: index.html, openapi.json, docs.css)   adminui/ (dist/)
    ├── web/admin/         React admin app (src/pages: Dashboard, Settings, Markets,
    │                      TestSearch, Tools, Clients, Admins, Audit)
    ├── deploy/            bootstrap-vm.sh, deploy.sh, searxng/settings.yml
    ├── Dockerfile  docker-compose.yml  compose.prod.yml  compose.dev.yml
    ├── .env.example       every setting, with comments
    └── README.md          developer reference
```

5.3 Where to look for...

Question	File
Which URL does what?	`internal/api/router.go` (`NewRouter`), `internal/api/admin.go` (`NewAdminRouter`)
How is a login token / API key checked?	`internal/api/access.go` (`requireCaller`), `internal/auth/jwt.go`, `internal/apikeys/apikeys.go`
When is a search counted?	`internal/metering/metering.go` (`search`)
How is a shop's cache key built?	`internal/cache/cache.go` (`key`, `resultsVersion = "v7"`)
How many Apify runs at once?	`internal/queue/queue.go` (`DefaultLimits`), settings `QUEUE_*`
How does a job fill in per shop?	`internal/api/jobwork.go` (`jobWork`), `internal/cache/cache.go` (`SearchProgress`)
When is `first_results` sent?	`internal/api/jobwork.go` (`fastDone`, `jobHooks.progress`)
How is a webhook signed?	`internal/webhook/webhook.go` (`Sign`, `Verify`)
Shop time limits and groups	`internal/config/config.go` (`shopDefaults`, `ShopRule`)
Product links per shop	`internal/markets/markets.json` (`product_pages`)
What runs at start / stop?	`cmd/owis_find_deal_engine/main.go` (`run`)

6. Life of the server process

6.1 Start

`run` does, in order:

1. Logger (JSON lines on stdout, `LOG_LEVEL`).
2. **Settings.** With `DATABASE_URL`: connect to Postgres, run migrations, on the first start copy the runtime settings found in `.env` into the database (author `import:env`), then read them; warn about runtime keys in `.env` that differ from the database. Add `ADMIN_EMAIL` as first admin if the list is empty. Without a database: everything from the environment.
3. **Login keys:** download Firebase's JWKS (refreshed hourly).
4. **Shared parts** (kept across settings changes): usage recorder (flushed to Postgres every 30 s), API-key authenticator (key cache 10 s, last-use saved every minute), Redis connection, usage store, cache store, **job runner**, **queue**, webhook secrets and sender.
5. `buildApp`: from the settings, build the providers (SerpApi engines, Apify scrapers, web chain), mark shops that are `off`, the search service with per-shop time limits, the cache front (with the shared queue), metering, rate limiters and the router. Logs one `shop rules` line per shop and one `apify scraper enabled` line per Apify shop.
6. **Queue runner** set to "the current app's cache" and the queue started (it immediately picks up tasks a stopped server left behind); `search queue` log line.
7. `api.ResumeJobs`: asynchronous jobs a stopped server left `running` are resumed.
8. Settings watcher started; admin router built (with a database); HTTP server listening on `:3002`; `server started` log line with the version (commit).

6.2 Settings change without restart

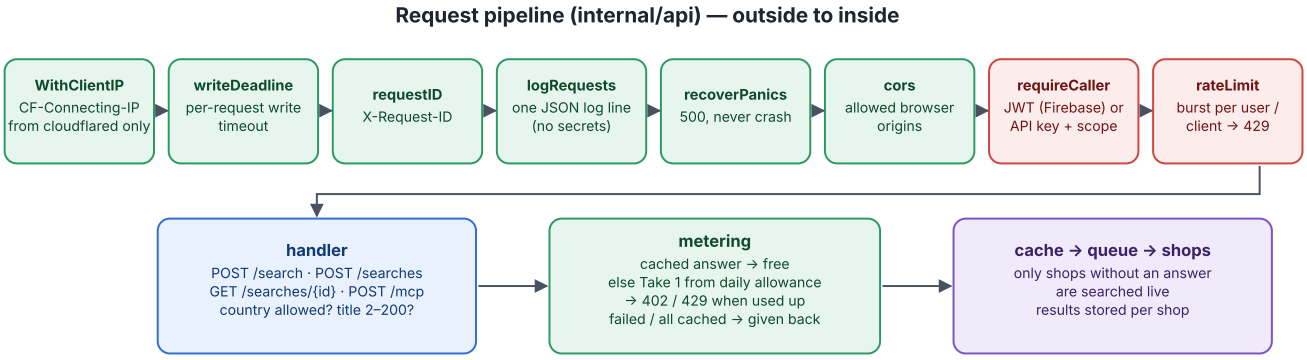
Every `SETTINGS_POLL_INTERVAL` (10 s) the watcher (`settings.Manager.Watch`) checks the settings version in Postgres. When it changed, `buildApp` builds a **new** provider set, cache front and router, and swaps it in atomically. Requests already running finish on the old one, which is closed afterwards. The queue's caps are updated (`SetOptions`) and the queue's runner always uses the current app. Log: `settings applied` with the version.

6.3 Stop (deploy, `docker restart`, SIGTERM)

1. Stop accepting HTTP requests; running requests get `SHUTDOWN_TIMEOUT` (10 s).
2. **Jobs:** with Redis they are stopped at once and **left** `running` with their answer so far (log `search job left for the next server start`); the cancel cause `search.ErrResumeLater` keeps them counted.
3. **Queue:** running shop searches are cut off and **put back in front of the queue** with attempt 2 (log `queue: task cut off, queued again`).
4. Cache background refreshes get the rest of the time; usage counters are flushed to Postgres; `server stopped cleanly`.

The next start resumes both (part 13).

7. The request pipeline: identity, limits, allowance



7.1 Middleware (outside to inside)

Step	Code	What it does
`WithClientIP`	`api/clientip.go`	caller's address: `CF-Connecting-IP` only from `TRUSTED_PROXIES`
`writeDeadline`	`api/middleware.go`	per-request write timeout (follows the search timeout)
`requestID`	`api/middleware.go`	`X-Request-ID` (yours, if valid, or a new one)
`logRequests`	`api/middleware.go`	one JSON log line per request: method, path, status, duration, request id, ip, user — never tokens or keys
`recoverPanics`	`api/middleware.go`	a bug answers `500`, the server keeps running
`cors`	`api/middleware.go`	browser origins from `CORS_ALLOWED_ORIGINS`
`requireCaller`	`api/access.go`	login token **or** API key (see below), scope check
`rateLimit`	`api/middleware.go`, `internal/ratelimit`	burst limit per user / client → `429 rate_limited`

7.2 Login tokens (app users)

`auth.Verifier.Verify`: the token's signature is checked **locally** with Firebase's public keys (JWKS, cached and refreshed); then `iss` (= `AUTH_ISSUER`), `aud` (= the Firebase project id), `exp`, `iat`. The user id is `sub`; the plan comes from the claim `AUTH_PLAN_CLAIM` (`plan`), else `DEFAULT_PLAN`. The sign-in method (`firebase.sign_in_provider`) must be in `AUTH_ALLOWED_SIGN_IN` (default `google.com,apple.com,password`): **anonymous accounts are refused** (`401`).

7.3 API keys (servers, agents)

`apikeys.Authenticator.Authenticate`: a key looks like `ofd_live_` + 40 random characters (200 bits). The server hashes it (SHA-256) and looks the hash up in Postgres (cached 10 s). It must belong to a client that is not revoked or expired, from an allowed address (`ip_allow`), with the scope the route needs (`search` or `countries`), and for searches an allowed country. The caller then acts as user `client:<id>` with its own daily quota and burst limit.

7.4 The daily allowance (metering)

`metering.Service` is the one place that decides what a search costs:

1. **Every shop answered from the cache → free** (not counted), even when the allowance is used up.
2. Otherwise **take one unit** from the caller's daily counter in Redis (`usage:<user>:<YYYYMMDD>`, UTC day): app users by plan (`PLANS`, e.g. `free:20,pro:500`), API clients by their own `daily_quota`. Used up → `402 payment_required` (free plan: show "upgrade") or `429 quota_exceeded` (paid plan / API client).
3. Run the search. **Give the unit back** when it failed or turned out fully cached (another request filled it meanwhile) — **except** when it was stopped by a server restart to be resumed (cause `ErrResumeLater`): then it stays counted, once.

A search is counted **once**, however many shops it searches. A caller who joins a run another caller started is counted too. The asynchronous search checks the allowance **before** creating the job (`Check`), so a used-up allowance is refused at once.

7.5 Limits at a glance

Limit	Value	Where
Search burst per user / client	`RATE_LIMIT_RPS` / `RATE_LIMIT_BURST` (default 1/s, burst 5); API clients: their own	`429 rate_limited`
Reading jobs + MCP messages	5/s, burst 20 per caller	separate from the search limit
Asynchronous jobs running	10 per caller, 200 per server	`429 too_many_jobs`
Admin API	10/s per IP before sign-in, 5/s per admin; test search ~1 per 6 s (burst 5)	
Title length	2–200 characters	`400 invalid_request`

8. The search engine: countries, shops, sources

8.1 The catalog

`markets.json` (embedded in the binary; editable in the admin panel → *Countries & shops*, stored as the `MARKETS` setting) has two parts:

- **markets** — per shop: name, the shop's own search link (`https://{host}/s?k={query}`), and the product-page rules (regex on the path, e.g. Amazon `/dp/<ASIN>`) used to recognise and clean product links;
- **countries** — per country: name, currency, region, language, optional `shopping_region`, and the list of shops with their site (`amazon.com` / `amazon.sa`).

8.2 Rules per shop

Each shop has exactly **one source**, a **time limit** and a **group** (`config.ShopRule`):

Setting	Values	Meaning
`SHOP_<SHOP>_SOURCE`	`serpapi` (Amazon, eBay), `apify`, `web`, `off`	who searches the shop; `off` = not searched, status `off`
`SHOP_<SHOP>_TIMEOUT`	e.g. `15s`, `90s`	how long one search of that shop may take (counted from when its run **starts**, not while it waits in the queue)
`SHOP_<SHOP>_GROUP`	`fast`, `slow`	`first_results` webhook is sent when the fast shops are done

Defaults (owner, 2026-11-09): Amazon / eBay 15 s fast, AliExpress 60 s slow (production set to **90 s**), Temu 45 s slow, SHEIN 30 s slow (production: **off**). Without `SHOP_<SHOP>_SOURCE`, the old lists decide (`APIFY_MARKETS` → apify, `SERPAPI_MARKETS` → serpapi, else web).

There is **no backup source**: a shop that fails or runs out of time shows its last known results (part 9), else `error`.

8.3 The sources

Source	Code	How it works	Cost
SerpApi Amazon	`providers/serpapi/amazon.go`	SerpApi's Amazon Search API on the country's Amazon; direct `/dp/<ASIN>` links, price, rating, image	one SerpApi search (`SERPAPI_SEARCH_COST_USD`)
SerpApi eBay	`providers/serpapi/ebay.go`	SerpApi's eBay Search API	one SerpApi search
Apify	`providers/apify/apify.go`	runs the shop's scraper (`APIFY_<SHOP>_ACTOR`, input template `APIFY_<SHOP>_INPUT` with `{{query}}`) with `run-sync-get-dataset-items`; sends `maxItems`, `maxTotalChargeUsd`, the run **timeout = shop limit − `APIFY_STOP_MARGIN`** (5 s; AliExpress 90 → 85 s) and optionally `memory` (`APIFY_<SHOP>_MEMORY_MB`)	per run (`APIFY_<SHOP>_RUN_COST_USD`; AliExpress ≈ $0.025, Temu ≈ $0.10)
web	`search/fallback.go` + `providers/searxng`, `providers/serpapi`	the chain `SEARCH_PROVIDERS` (e.g. `searxng,serpapi`), site-restricted queries; each provider gets `PROVIDER_ATTEMPT_TIMEOUT`; a **circuit breaker** skips a provider after `PROVIDER_FAILURE_THRESHOLD` failures for `PROVIDER_COOLDOWN`	SearXNG free; SerpApi per search

8.4 One shop's search, step by step

`search.Service.Search` (called by a queue task with one shop):

1. Normalise the title (spaces), check 2–200 characters and the country.
2. Group the shop's target into a provider call; apply the shop's time limit to the call.

3. Call the provider; keep only **relevant** products (`search.Relevant`: the title's words must match), map links to clean product pages (`markets.Target.ProductLink`), or to the shop's search (`link_type: "search"`) when no product page is known.
4. **Currencies**: `$`, `US$` → `USD`, `€` → `EUR`, `£` → `GBP` (`NormalizeCurrency`).
5. **Dedupe** the same product within a shop (cheapest offer kept), sort by rank then the country's shop order, keep `RESULTS_PER_MARKET` (10) per shop.
6. Return the products and the shop's status: `ok`, `no_results` or `error`, with source, time taken and the fetch date.

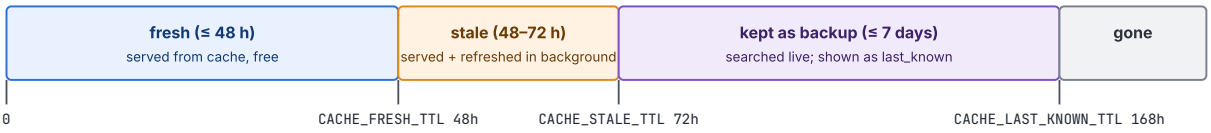
8.5 Statuses a shop can have in an answer

Status	Meaning
`ok`	results found
`no_results`	searched fine, nothing matched
`error`	failed or ran out of time, and no recent results to show
`last_known`	failed or ran out of time; its **latest results** (≤ 7 days, dated in `shops.<shop>.fetched_at`) are shown
`off`	the shop is turned off
`pending`	asynchronous search only: still being searched

A search fails (`502 upstream_error`, job `failed`) only when **every** shop is `error` or `off`.

9. The cache

One shop's results in the cache (key shop:v7:<shop>:<hash>) — age since fetched



- A shop that found nothing is fresh only CACHE_EMPTY_TTL (30 min). Failures are never stored.
- "refresh": true forces a live search at most once per CACHE_MIN_REFRESH (10 min) per shop.
- Countries share entries when the shop request is the same (AliExpress, Temu everywhere; amazon.com for usa + jor).
- Key = shop + site + source setup + normalized title (+ region for web shops). CACHE_GENERATION drops everything.

Code: `internal/cache/cache.go`, stores in `internal/cache/store.go` (Redis, or an in-memory LRU without Redis).

Per shop, not per search. Each shop's results are stored on their own under `shop:v7:<shop>:<hash>`. The hash covers the cache generation, the shop and its site, the **setup of its source** (source, actor, input, max items, currency, results per shop…), the region for web shops (and scrapers whose input contains `{{country}}`), and the lower-cased title. Consequences:

- A search runs live **only for the shops without an answer**; the others come from the cache.
- **Countries share** a shop's results when the request to the shop is the same: AliExpress and Temu in every country, `amazon.com` for `usa` and `jor`. (That is why a title searched in one country is often instant in another.)
- Changing a shop's source or settings starts new entries **for that shop only**. `CACHE_GENERATION` (admin → *Cache & sources* → bump) makes every entry a miss.

Freshness (settings `CACHE_*`):

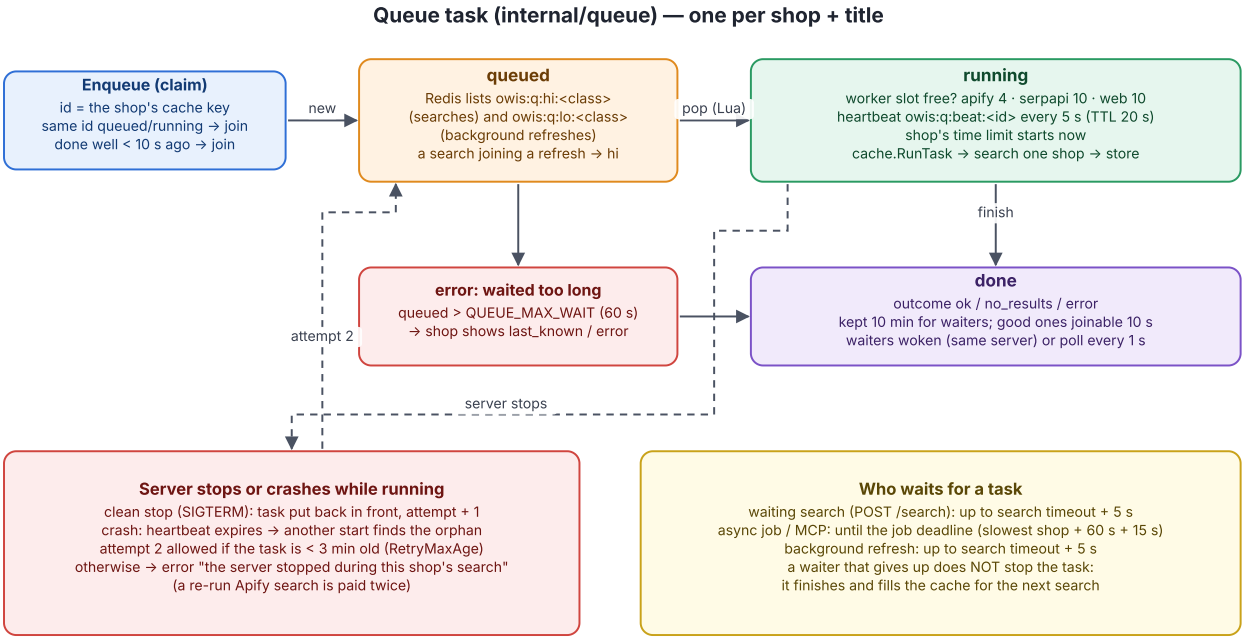
Age of a shop's results	Behaviour	`X-Cache`
< `CACHE_FRESH_TTL` (48 h)	served from the cache	`HIT`
up to `CACHE_STALE_TTL` (72 h)	served at once, **refreshed in the background** for the next caller (queue tasks behind searches, at most `CACHE_MAX_BACKGROUND_REFRESH` at once)	`STALE`
older	searched live	`MISS`
up to `CACHE_LAST_KNOWN_TTL` (7 days)	kept as **backup**: shown as `last_known` when the live search fails	

- A shop that found nothing is fresh only `CACHE_EMPTY_TTL` (30 min).
- **Failures are never stored.**
- `"refresh": true` (pull to refresh) forces a live search, at most once per `CACHE_MIN_REFRESH` (10 min) per shop.
- The answer's `fetched_at` is the **oldest** shop's date; `shops.<shop>.fetched_at` gives each shop's date — the app can show "prices updated 3 h ago".
- Admin → *Cache & sources* → **forget** a title + country makes its next search live (`Cache.Forget`).

`SearchProgress` — the heart of the asynchronous search: it builds the plan (which shops are cached, which need a live search), reports the answer so far at once (cached shops `ok`, live ones `pending`), sends each live shop to the queue, and reports again each time a shop finishes. A shop not done when the caller's wait ends shows `last_known` / `error` — but its task goes on and stores the result for the next search.

`RunTask` — what a queue worker runs: search one shop (`search.Service.Search` with that one shop) and store its entry (`ok` / `no_results` only).

10. The queue



Code: `internal/queue/queue.go` (engine) and `internal/queue/store.go` (Redis with Lua scripts, or memory). Built in step 14b part 3.

Why a queue? Paid sources must not be flooded (Apify's free plan has 8 GB = four 2048 MB runs at once), the same shop + title must not be paid twice when several people search it at the same moment, and a search cut off by a restart should go on.

How it works

1. **Task = one shop's live search for one title.** Its id is the shop's cache key, so the same shop and title (also from another country when the entry is shared) is **the same task**.
2. **Enqueue / claim** (Lua `claimScript`): if a task with that id is queued or running — or finished well less than 10 s ago — the caller **joins** it. Otherwise it is queued in its class's lane: `owis:q:hi:<class>` for searches, `owis:q:lo:<class>` for background refreshes. A search that joins a queued refresh moves it to the front lane.
3. **Classes and caps** (`queue.ClassOf`): `apify` (`QUEUE_APIFY_MAX_RUNS`, 4), `serpapi` (`QUEUE_SERPAPI_MAX_RUNS`, 10), `web` (`QUEUE_WEB_MAX_RUNS`, 10). One dispatcher per class takes the next task when a slot is free (`popScript`: searches first).
4. **Queue wait limit.** A task that waited longer than `QUEUE_MAX_WAIT` (60 s) for a slot ends as an error ("waited … in the queue") — the shop shows `last_known` / `error`.
5. **Running.** The worker calls the current app's `Cache.RunTask`; the shop's time limit starts now. A **heartbeat** key `owis:q:beat:<id>` is renewed every 5 s (lives 20 s).
6. **Done.** The outcome is saved in `owis:q:task:<id>` (kept 10 min); waiters on the same server are woken at once, others check every second.
7. **Restarts.** A clean stop puts running tasks back **in front** with attempt 2. After a crash the heartbeat expires and the next server (it checks at start and every 10 s, `orphanScript`) finds the orphan. A task runs **again at most once**, and only if it is younger than 3 minutes; otherwise it ends as an error. A re-run Apify search is paid twice (Apify keeps running the cut-off run).

Who waits, and how long

Waiter	Waits up to	Then
Waiting search `POST /api/v1/search`	search timeout + 5 s	shop shows `last_known` / `error`; task goes on

Waiter	Waits up to	Then
Asynchronous job / MCP	the job deadline = slowest shop + `QUEUE_MAX_WAIT` + 15 s (production 90 + 60 + 15 = 165 s)	job ends
Background refresh	search timeout + 5 s	

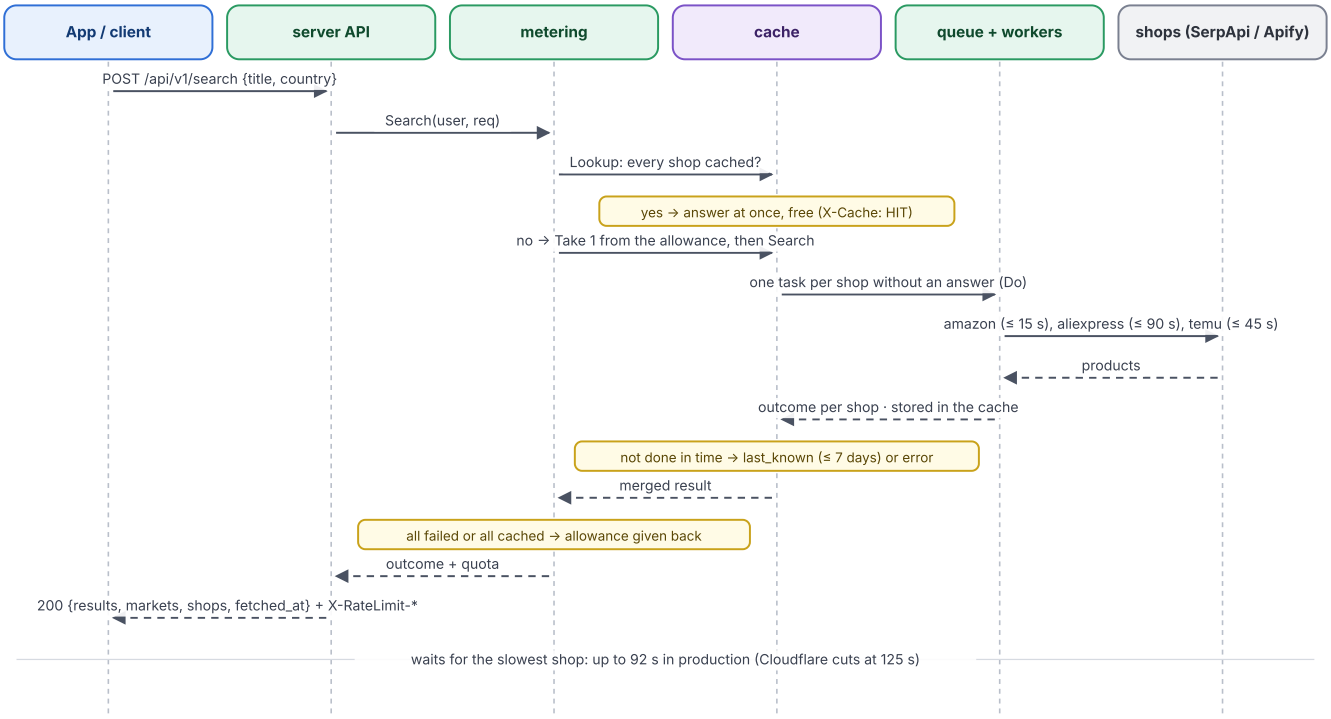
Without Redis the queue lives in memory (one server; lost on restart). The admin test search does **not** use the queue (it asks each source directly).

11. Waiting search vs asynchronous search

Both run the same search, with the same metering and cache. They differ in how the caller gets the answer.

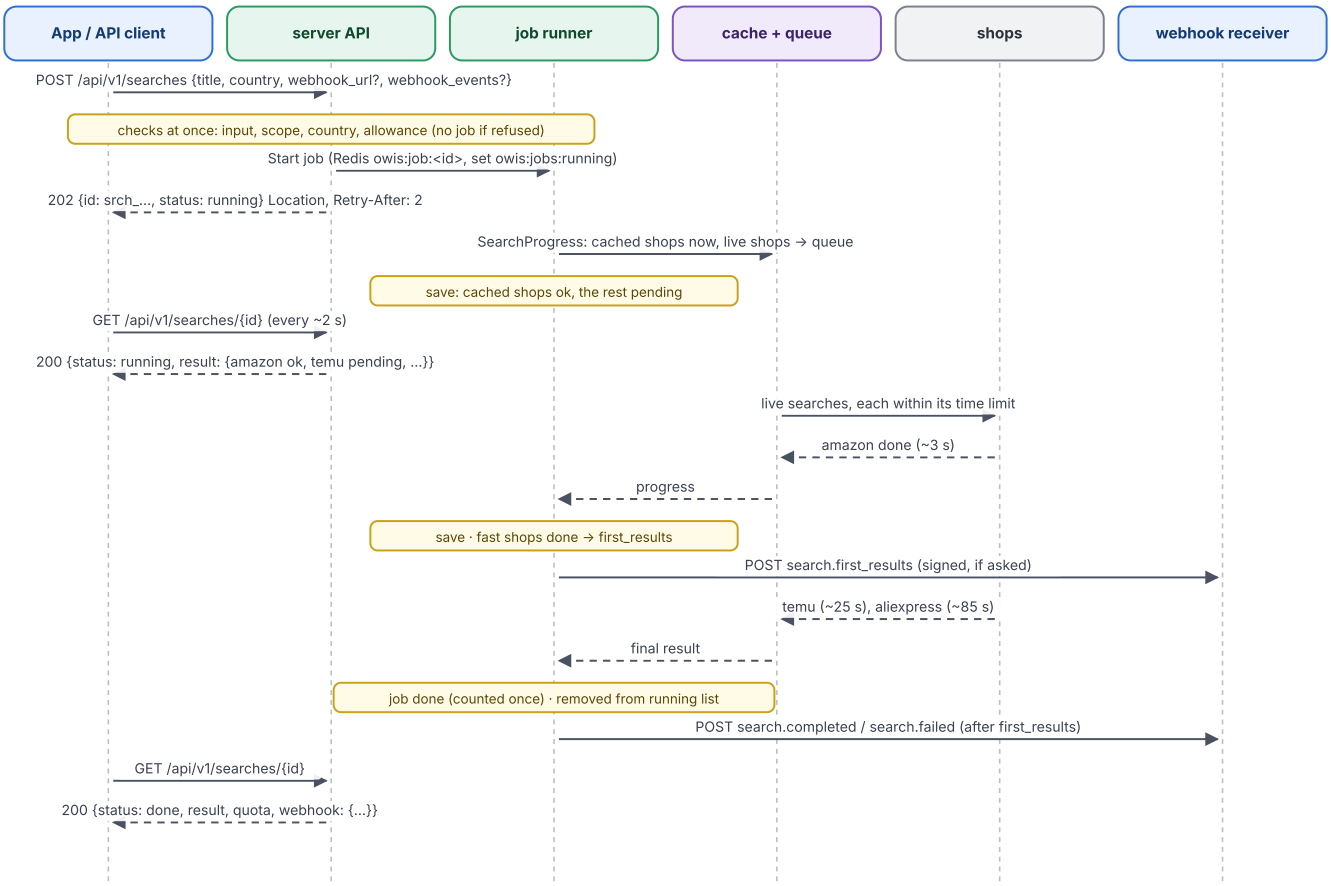
	Waiting (synchronous)	Asynchronous
Endpoint	`POST /api/v1/search`	`POST /api/v1/searches` → `GET /api/v1/searches/{id}`
Answer	`200` with the full result when the slowest shop is done (≤ 92 s)	`202` at once with a job id; the job fills in per shop
For	simple API clients (decision A: kept)	**the app** (polling every ~2 s), API clients (polling or webhooks), MCP
Shows fast shops early	no	**yes** (`pending` for the others)
Webhooks	no	API keys only: `first_results` , `completed`
Survives a server restart	no (the request ends)	**yes** (resumed)
Admin test search	separate, synchronous, no cache / queue / metering	—

11.1 Waiting search



Code: `api/handlers.go` (`search`) → `metering.Service.Search` → `cache.Search` (= `SearchProgress` with the waiting limit, no progress) → queue → shops. Response headers: `X-Cache` (`HIT` , `STALE` , `MISS`), `X-Plan` , `X-RateLimit-Limit` , `X-RateLimit-Remaining` , `X-RateLimit-Reset` .

11.2 Asynchronous search



Code: `api/searches.go` (`startSearch`, `getSearch`), `api/jobwork.go` (`jobWork`), `internal/jobs/jobs.go` (`Runner`).

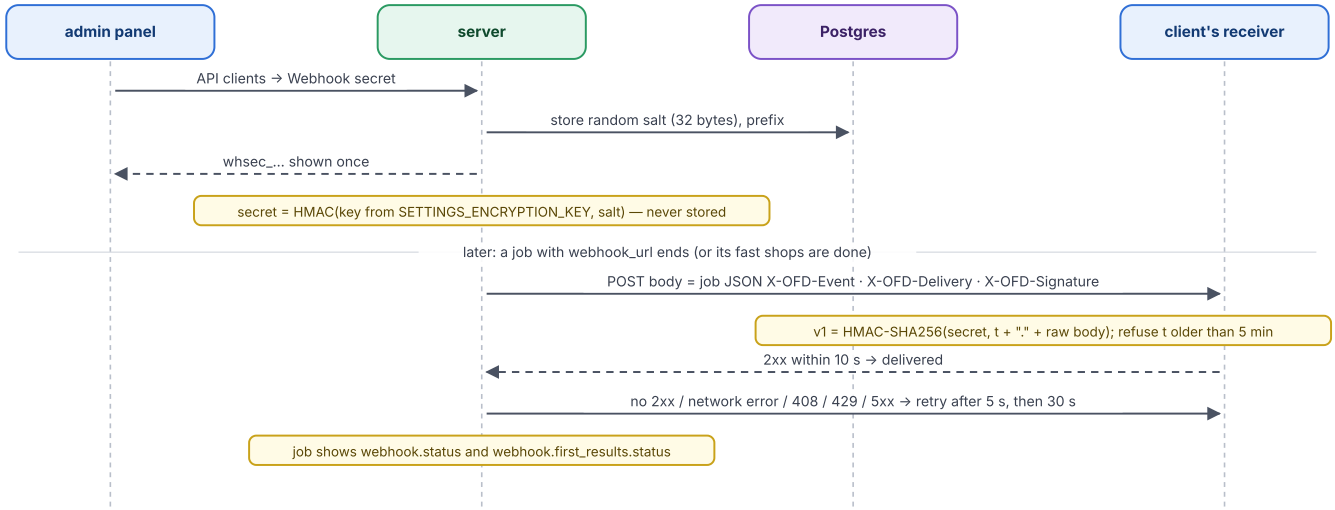
1. **`POST /api/v1/searches`** `{"title","country","refresh"?,"webhook_url"?,"webhook_events"?}`. Checked at once, before any job: JSON, title, country, scope, allowed country, webhook URL / secret, and the **allowance** (`metering.Check`).
2. A job `srch_<26 chars>` is saved in Redis (`owis:job:<id>`, listed in `owis:jobs:running`) and started in the background. If the whole answer is cached, the `202` already carries it (`status: done`).
3. `202 Accepted`, `Location: /api/v1/searches/<id>`, `Retry-After: 2`.
4. The job runs `metering.SearchProgress`: each time a shop finishes, the job is **saved with the answer so far**.
5. **`GET /api/v1/searches/{id}`** (only the caller who started it; others get `404`): `status` `running` (with `result` holding the shops done and `pending` for the rest), `done` (full `result`, `quota`) or `failed` (`error`: the HTTP status and code the waiting search would have given, e.g. `502 upstream_error`, `504 timeout`, `503 job_lost`). Jobs are kept 1 hour after they finish.

Example while running:

```
{"id":"srch_…","status":"running",
 "request":{"title":"wireless mouse","country":"usa","refresh":false},
 "created_at":"…","finished_at":null,
 "result":{"markets":{"amazon":"ok","ebay":"ok","aliexpress":"pending","temu":"pending","shein":"off"},
           "results":[{"market":"amazon","title":"…","price":19.99,"currency":"USD", …}], …}}
```

For the app: start the search, show what is there, poll every 2 s (the `Retry-After` hint) until `status` is not `running`; show a shop's "updated 3 days ago" for `last_known`.

12. Webhooks



Code: `internal/webhook/webhook.go`, `api/jobwork.go` (`jobHooks`), `api/admin_clients.go` (secret).

Setup (once per API client). Admin panel → *API clients* → the client → **Webhook secret**. The server stores a random 32-byte **salt** and shows `whsec_…` **once**. The secret itself is never stored: it is `HMAC(key derived from SETTINGS_ENCRYPTION_KEY, salt)`, so a copy of the database alone cannot sign. A new secret replaces the old one at once. Without a secret, `webhook_url` is refused (`400 webhook_not_configured`).

Ask for webhooks on `POST /api/v1/searches`:

```
{"title":"usb c hub","country":"jor",
 "webhook_url":"https://example.com/hooks/ofd",
 "webhook_events":["first_results","completed"]}
```

Event (`X-OFD-Event`)	Sent when	`X-OFD-Delivery`
`search.first_results`	(asked with `first_results`) the **fast** shops are done while others are still `pending` — body `status: running`. No fast shop in the country: when the first shop is done. Not sent when nothing is left to wait for (`skipped`).	`<job id>.first_results`
`search.completed`	the job is done	`<job id>`
`search.failed`	the job failed	`<job id>`

`webhook_events` defaults to `["completed"]` (old clients see no change). `first_results` always arrives **before** `completed`.

Each call: `POST` with the job JSON (as `GET` shows it, without `webhook`), headers `Content-Type: application/json`, `X-OFD-Event`, `X-OFD-Delivery`, and

```
X-OFD-Signature: t=<unix seconds>,v1=<hex HMAC-SHA256(secret, "<t>." + raw body)>
```

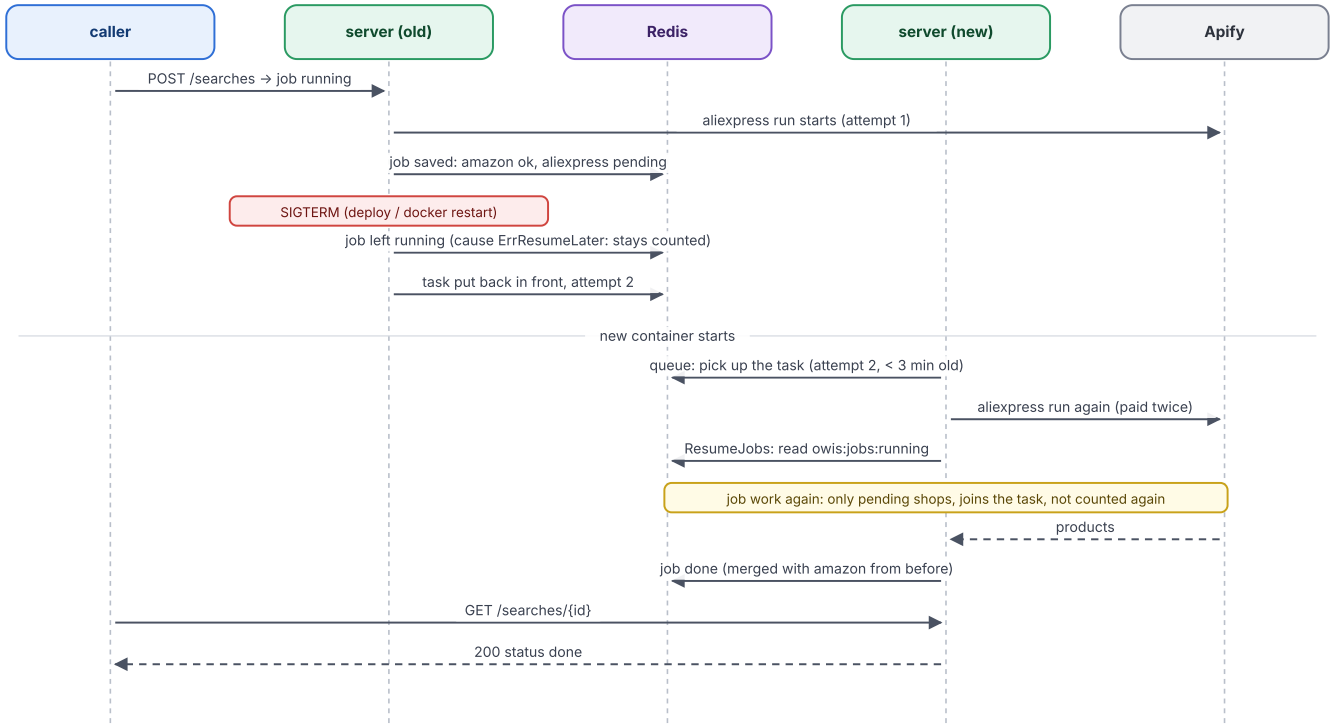
Rules: only `https` to public addresses (internal ones refused, also after DNS; `WEBHOOK_ALLOW_PRIVATE=true` only for local tests); redirects not followed; any `2xx` within 10 s = delivered; network error, `408`, `429`, `5xx` → retried after 5 s and 30 s (3 attempts). The job shows the outcome: `webhook.status` (`pending`, `delivered`, `failed`, `not_requested`) and `webhook.first_results.status` (`pending`, `delivered`, `failed`, `skipped`).

Checking a webhook (receiver side, Python):

```
import hashlib, hmac, time
def check(secret: str, header: str, raw_body: bytes, max_age=300) -> bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    t, v1 = int(parts["t"]), parts["v1"]
    if abs(time.time() - t) > max_age:
        return False                      # old: maybe a replay
    want = hmac.new(secret.encode(), f"{t}.".encode() + raw_body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(want, v1)  # constant time
```

Use `X-OFD-Delivery` to ignore repeats.

13. Restarts, failures and how the service recovers



A restart during an asynchronous search (deploy, `docker restart`, VM reboot) — tested in production on 2026-11-11:

1. The old server stops its jobs at once; each stays `running` in Redis with its answer so far, and stays **counted** (cause `ErrResumeLater`).
2. Running shop tasks go back in front of the queue with attempt 2.
3. The new server's queue picks the task up again (if < 3 min old) — the Apify run is paid a second time.
4. `api.ResumeJobs` reads `owis:jobs:running`; each job searches **only its pending shops** (joining their tasks), merges them with what it had, and finishes; webhooks are sent (the secret is derived again from the client's salt). The caller just keeps polling.
5. Log lines: `search job left for the next server start`, `queue: task cut off, queued again`, `search jobs resumed after the restart`.

What happens when...

Event	Effect	What the caller sees
One shop fails	other shops answer	that shop `last_known` (≤ 7 days) or `error`
A shop is slower than its limit	cut at the limit (Apify stopped 5 s earlier)	`last_known` / `error`
Every shop fails	search fails, unit given back	`502 upstream_error` / job `failed`
Too many paid runs at once	tasks wait for a slot (≤ 60 s)	`pending` longer; then `last_known` / `error`
Same title searched by many	one task per shop, joined	everyone gets the same results; each counted once
Redis down	allowance cannot be checked: live searches refused (fail closed)	`503 service_unavailable`
Postgres down	settings changes, API-key checks and the admin panel unavailable (keys already checked stay cached 10 s)	API keys `503 service_unavailable`; app users unaffected
Server crash	jobs lost after their deadline unless resumed; tasks re-run once by heartbeat	job `failed` `503 job_lost`, or resumed

Event	Effect	What the caller sees
A provider keeps failing (web chain)	circuit breaker skips it for `PROVIDER_COOLDOWN`	slower or `last_known`
Monthly estimated cost over budget	`ERROR` log + audit entry `budget.alert`; nothing blocked	—

14. AI agents (MCP)

Code: `internal/api/mcp.go`. Endpoint `POST /mcp` (Model Context Protocol over Streamable HTTP), **API keys only** (`Authorization: Bearer ofd_live_…`).

Tool	Scope	Arguments	Result
`list_countries`	`countries`	none	countries this key may use, currency, shops — free
`search_products`	`search`	`title`, `country`, `refresh`?	starts an asynchronous search, **waits up to 15 s**, returns what is ready + `status` (`done` / `running`), `pending`, `job_id`, `quota`; metered like a search
`get_search_results`	`search`	`job_id`	the rest of that search; waits up to 15 s while it runs; free; only the key that started it

- Results come as `structuredContent` (with an output schema) and as the same JSON in a text block; tool failures are tool results with `isError: true` and `{"error":{"code","message"}}` so the agent can read them.
- Protocol versions: `2026-07-28` (stateless, headers `MCP-Protocol-Version`, `Mcp-Method`, `Mcp-Name`) and `2025-03-26` / `2025-06-18` / `2025-11-25` (with `initialize`). One JSON answer per request, no sessions, no SSE.
- Limits: 5 messages/s per client; `search_products` also uses the client's search burst limit and daily quota.
- **Connect Claude Code:**

```
claude mcp add --transport http owis-find-deal https://globalprodsearch.dawirli.com/mcp --header "Authorization: Bearer ofd_live_…"
```

When every shop finishes within 15 s (as on 2026-11-11, AliExpress 8 s), `search_products` answers `done` directly.

15. Public API reference

Base URL `https://globalprodsearch.dawirli.com`. Full description: `/api/v1/openapi.json` (OpenAPI 3.1) and the developer page `/docs/`.

Method and path	Auth	What
`GET /api/v1/health`	public	`{"status":"ok"}`
`GET /api/v1/countries`	public	countries and their shops
`POST /api/v1/search`	token or key (`search`)	waiting search
`POST /api/v1/searches`	token or key (`search`)	asynchronous search → `202` + job
`GET /api/v1/searches/{id}`	the job's caller	the job
`POST /mcp`	key	MCP tools
`GET /api/v1/openapi.json`, `GET /docs/`	public	API description, developer page
`/admin/…`	admin (blocked on the public address)	admin web app and admin API

Search request: `{"title":"usb c hub","country":"jor","refresh":false}`

Search answer (also the job's `result`):

```
{
  "query": "usb c hub", "country": "jor",
  "results": [
    {"market":"amazon","title":"…","link":"https://www.amazon.com/dp/B0…","thumbnail":"…",
     "price":24.99,"currency":"USD","position":1,"provider":"serpapi","extensions":["4.5 stars"]}
  ],
  "markets": {"amazon":"ok","aliexpress":"ok","temu":"last_known","shein":"off"},
  "shops": {
    "amazon":{"status":"ok","source":"serpapi-amazon","fetched_at":"…","took_ms":2210,"cached":false},
    "temu":{"status":"last_known","source":"apify","fetched_at":"2026-11-08T…","took_ms":45002,"cached":true}
  },
  "took_ms": 8314, "fetched_at": "2026-11-08T…", "cached": false, "stale": false
}
```

`link_type: "search"` on a product means the link opens the shop's own search (no product page known).

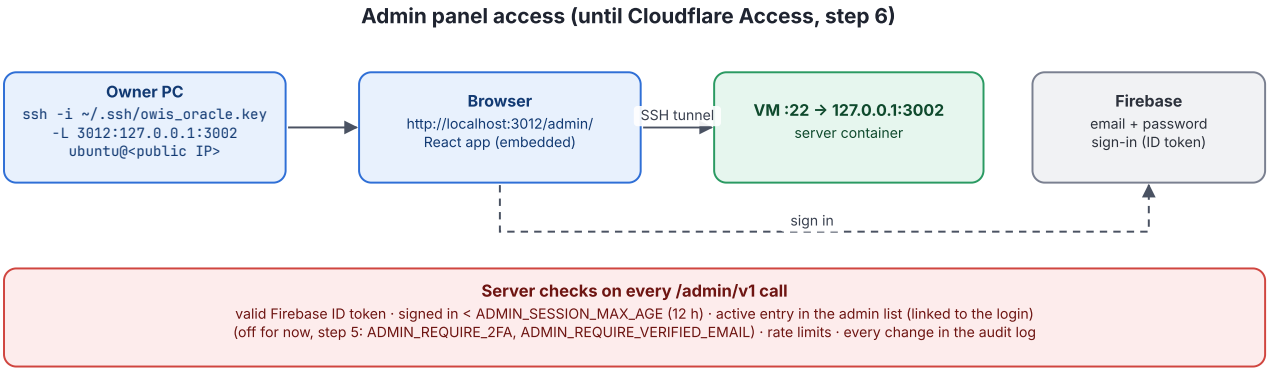
Errors always look like `{"error":{"code":"…","message":"…"}}`:

Status	Code	Meaning
400	`invalid_request`	bad JSON, unknown field, title not 2–200, bad webhook URL / events
400	`unsupported_country`	unknown country
400	`webhook_not_configured`	`webhook_url` given, client has no webhook secret
401	`unauthorized`	missing / invalid / expired token or key; refused sign-in method
402	`payment_required`	free plan's daily searches used up
403	`forbidden`	key lacks the scope or address not allowed
403	`country_not_allowed`	key limited to other countries
404	`not_found`	no such job (or someone else's, or older than 1 h)
429	`rate_limited`	too fast (see `Retry-After`)
429	`quota_exceeded`	paid plan's / API client's daily searches used up
429	`too_many_jobs`	10 asynchronous searches already running
502	`upstream_error`	every shop failed

Status	Code	Meaning
503	`service_unavailable`	Redis unreachable (fails closed) or busy
500	`internal_error`	bug

16. Admin panel and admin API

16.1 Getting in



The admin panel is blocked on the public address until Cloudflare Access (step 6). Today:

```
ssh -i ~/.ssh/owis_oracle.key -L 3012:127.0.0.1:3002 ubuntu@<public IP>
```

then open `http://localhost:3012/admin/` and sign in with your Firebase email and password. The server checks on every call (`internal/api/admin.go`): a valid Firebase ID token, signed in less than `ADMIN_SESSION_MAX_AGE` (12 h) ago, an **active entry in the admin list** (the first sign-in links your Firebase account to the entry). Two-factor login (`ADMIN_REQUIRE_2FA`) and verified email (`ADMIN_REQUIRE_VERIFIED_EMAIL`) exist in the code but are **off until step 5**.

16.2 Pages

Page	What you do there
Dashboard	searches per day (hits, stale, live, refused, errors), estimated cost per day, budget, sources per shop (calls, errors, avg time, cost), countries, fallbacks, API clients, top queries
Settings	every runtime setting grouped (Shops, Sources, Apify, Search, Queue, Cache, Plans…), with help texts; secrets write-only; changes saved together, validated as a whole, applied within 10 s; change history
Countries & shops	overview of the catalog and a JSON editor (`markets.json` format), validated
Test search	asks each shop's source directly (no cache, queue or quota), within the shop's limit: products, time, error, estimated cost; option to try every web provider
Cache & sources	server status (version, settings version), forget a cached title + country, bump the cache generation, circuit breaker state per source
API clients	create (name, owner, scopes, daily quota, rate, countries, IP allow-list, expiry), change, **new key with grace period**, stop a key, revoke the client, **webhook secret**; the key / secret is shown once with a copy button and a `curl` example
Admins	add, disable, enable, unlink admins (never the last active one)
Audit log	every change and test search: who, when, IP, request id (never secret values)

16.3 Admin API

`/admin/v1/…` with `Authorization: Bearer <admin's Firebase ID token>`; answers `Cache-Control: no-store`.

Call	What
`GET /admin/v1/me`, `GET /admin/v1/status`	the signed-in admin; build, settings version, markets source, cache generation
`GET` / `PATCH /admin/v1/settings`, `GET /admin/v1/settings/history`	settings (`{"changes": {"KEY":"value","OTHER":null}}`, `null` = default)

Call	What
`GET` / `PUT` / `DELETE /admin/v1/markets`	the catalog
`GET` / `POST /admin/v1/admins`, `POST /admin/v1/admins/{email}/{disable|enable|unlink}`	admins
`GET /admin/v1/audit`	audit log
`POST /admin/v1/tools/test-search`	`{"title","country","markets":[…],"all_sources":false}`
`POST /admin/v1/tools/cache/forget`, `POST /admin/v1/tools/cache/bump`, `GET /admin/v1/tools/providers`	cache and sources
`GET /admin/v1/usage?days=30&top=20`	dashboard data
`GET` / `POST /admin/v1/clients`, `GET` / `PATCH /admin/v1/clients/{id}`	API clients
`POST /admin/v1/clients/{id}/rotate` `{"grace":"24h"}`, `…/revoke`, `…/keys/{key}/revoke`, `…/webhook-secret`	keys and secrets

16.4 Admin command line (in the container)

For setup and recovery (for example, every admin locked out):

```
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine settings list
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine settings set SHOP_ALIEXPRESS_TIMEOUT 90s
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine settings unset APIFY_ALIEXPRESS_MEMORY_MB
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine settings history SHOP_ALIEXPRESS_TIMEOUT
docker exec -i owis_find_deal_engine-server-1 /owis_find_deal_engine settings set SERPAPI_KEY - < serpapi.key
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine admins list
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine admins add someone@example.com
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine clients list
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine clients revoke 3
docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine audit settings.
```

(`VALUE` `-` reads one line from standard input — for secrets, so they stay out of the shell history.)

17. Settings reference

Two kinds (`internal/config/keys.go`):

- **bootstrap** — only in `.env` , fixed while the server runs: `PORT` , `DATABASE_URL` , `SETTINGS_ENCRYPTION_KEY` , `SETTINGS_POLL_INTERVAL` , `ADMIN_EMAIL` , `ADMIN_REQUIRE_2FA` , `ADMIN_REQUIRE_VERIFIED_EMAIL` , `ADMIN_SESSION_MAX_AGE` , `ADMIN_FIREBASE_API_KEY` , `ADMIN_FIREBASE_AUTH_DOMAIN` , `AUTH_*` , `REDIS_URL` , `SEARXNG_URL` , `*_BASE_URL` , `TRUSTED_PROXIES` , `WEBHOOK_ALLOW_PRIVATE` , `MARKETS_FILE` , `CACHE_MAX_ENTRIES` , `LOG_LEVEL` , `SHUTDOWN_TIMEOUT` ;
- **runtime** — in Postgres, changed in the admin panel or CLI, applied within 10 s without restart; copied from `.env` once at the first start. Every change is validated as the **whole** configuration it leads to; secrets are encrypted and write-only; every change is in the history.

Runtime settings (defaults; production differences in **bold**):

Group	Setting	Default	Meaning
Shops	`SHOP_<SHOP>_SOURCE`	from old lists	`serpapi` , `apify` , `web` , `off` (**shein** `off`)
	`SHOP_<SHOP>_TIMEOUT`	amazon/ebay 15s, aliexpress 60s, temu 45s, shein 30s	**aliexpress 90s**
	`SHOP_<SHOP>_GROUP`	amazon/ebay fast, others slow	
Queue	`QUEUE_APIFY_MAX_RUNS`	4	Apify runs at once (all searches)
	`QUEUE_SERPAPI_MAX_RUNS` / `QUEUE_WEB_MAX_RUNS`	10 / 10	
	`QUEUE_MAX_WAIT`	60s	longest wait for a free run
Apify	`APIFY_TOKEN`	—	secret
	`APIFY_MARKETS`	—	old list (**aliexpress,temu**)
	`APIFY_<SHOP>_ACTOR` , `_INPUT` , `_CURRENCY` , `_MAX_ITEMS` , `_RUN_COST_USD` , `_MEMORY_MB`	—	scraper, input with `{{query}}` , currency for bare prices, results per run, estimated cost, run memory
	`APIFY_MAX_ITEMS` / `APIFY_TIMEOUT` / `APIFY_MAX_CHARGE_USD`	10 / 60s / 0.20	defaults per run
	`APIFY_STOP_MARGIN`	5s	Apify stops this long before the shop's limit
SerpApi / web	`SERPAPI_KEY`	—	secret
	`SERPAPI_MARKETS` , `SERPAPI_ENGINE` , `SERPAPI_TIMEOUT`	amazon,ebay / google_shopping / 25s	old list, engine for web shops
	`SEARCH_PROVIDERS`	`searxng,serpapi`	web chain order
	`PROVIDER_ATTEMPT_TIMEOUT` , `PROVIDER_FAILURE_THRESHOLD` , `PROVIDER_COOLDOWN`	8s, 3, 1m	web chain + circuit breaker
	`SEARXNG_ONE_QUERY` , `SEARXNG_MAX_PAGES` , `SEARCH_COMBINED`	false, 2, true	web search details
Search	`SEARCH_TIMEOUT`	15s (raised to slowest shop + 2 s → **92s**)	waiting search
	`SEARCH_MAX_CONCURRENCY` , `RESULTS_PER_MARKET`	4, 10	
Cache	`CACHE_ENABLED`	true	
	`CACHE_FRESH_TTL` , `CACHE_STALE_TTL` , `CACHE_LAST_KNOWN_TTL`	48h, 72h, 168h	part 9

Group	Setting	Default	Meaning
	`CACHE_EMPTY_TTL`, `CACHE_MIN_REFRESH`, `CACHE_MAX_BACKGROUND_REFRESH`, `CACHE_GENERATION`	30m, 10m, 4, 0	
Plans	`PLANS`, `DEFAULT_PLAN`	`free:20,pro:500`, `free`	app users' daily live searches
	`RATE_LIMIT_RPS`, `RATE_LIMIT_BURST`	1, 5	per user
	`CORS_ALLOWED_ORIGINS`	empty	browser origins
Cost	`SERPAPI_SEARCH_COST_USD`, `BUDGET_MONTHLY_USD`, `BUDGET_ALERT_PERCENT`	0, 5, 80	estimates and budget
Catalog	`MARKETS`	built-in `markets.json`	edited in *Countries & shops*
Old	`CACHE_PARTIAL_TTL`, `APIFY_EMPTY_FALLBACK`	—	no effect since step 14b

The full list with comments is `.env.example`.

18. Data stores: Postgres and Redis

18.1 Postgres (`internal/store/migrations`)

Table	Holds
`schema_migrations`	applied migrations
`settings`	runtime settings (`key` , `value` — encrypted for secrets, `secret` , `updated_at` , `updated_by`)
`settings_history`	every change: old → new (secrets: only that they changed), who, when
`admins`	admin list: email, linked Firebase uid, disabled
`audit_log`	admin actions: actor, action, target, detail (JSON), outcome, request id, IP
`usage_daily`	per UTC day and kind: searches, source calls, fallbacks, cost, time (kept 400 days)
`query_daily`	most searched queries per country and day (kept 90 days, no user ids)
`api_clients`	API clients: scopes, daily quota, rate, countries, IP allow-list, expiry, revoked, webhook salt + prefix
`api_keys`	keys: client, prefix, **SHA-256 hash**, expiry (rotation grace), revoked, last used

Migrations `0001` … `0005` run automatically at start.

18.2 Redis keys

Key	Holds	Lifetime
`usage:<user>:<YYYYMMDD>`	daily allowance counter (user id or `client:<id>`)	48 h
`shop:v7:<shop>:<hash>`	one shop's cached results	up to 7 days
`owis:job:<id>`	an asynchronous job (JSON)	until 1 h after it ends
`owis:jobs:running`	set of running job ids (for resume)	—
`owis:q:task:<id>`	queue task: state, task JSON, outcome	15 min active, 10 min after done
`owis:q:hi:<class>` , `owis:q:lo:<class>`	queued task ids (searches, refreshes)	—
`owis:q:run`	running task ids (all servers)	—
`owis:q:beat:<id>`	heartbeat of a running task	20 s

Redis keeps data across restarts (AOF), has a 256 MB limit, and under memory pressure evicts keys **closest to expiry** first (`volatile-ttl`).

19. Usage, cost and budget

Code: `internal/stats`, `search/instrument.go`.

- Each server counts in memory and adds the counts to Postgres every 30 s and at shutdown (`usage_daily`, `query_daily`).
- **Costs are estimates**: `SERPAPI_SEARCH_COST_USD` per SerpApi call and `APIFY_<SHOP>_RUN_COST_USD` per scraper run (admin test searches included). The real bill is in the SerpApi and Apify consoles.
- **Budget**: this month's estimated cost against `BUDGET_MONTHLY_USD` (default $5 = Apify's free credit), `warning` from `BUDGET_ALERT_PERCENT` (80 %), `exceeded` above 100 %. Each level raises one alert per month: an `ERROR` log line and an audit entry `budget.alert`. **Nothing is blocked.**
- Per API client: searches split as for users, `cost_usd_estimate`, last search day — the base for billing partners later.

20. Command handbook: PC and VM

All PC commands are for **Git Bash** on Windows (they also work on macOS / Linux). Replace `<…>` with your values.

20.1 One-time on your PC

```
git clone https://github.com/owisdev/claude-ai-clould-repo.git
cd claude-ai-clould-repo/Owis_Find_Deal_Engine/server
```

```
export MSYS_NO_PATHCONV=1   # Git Bash: stops /paths being turned into Windows paths
```

20.2 Variables for testing production (each new Git Bash window)

```
export BASE=https://globalprodsearch.dawirli.com
export KEY=ofd_live_<your owner-test key>
```

App-user login token (valid 1 hour; `<WEB_API_KEY>` = Firebase web API key, same as `ADMIN_FIREBASE_API_KEY`). First time:

```
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=<WEB_API_KEY>" -H "Content-Type:
application/json" -d '{"email":"test1@example.com","password":"Test-pass-123","returnSecureToken":true}' | grep -o
'"idToken": *"[^"]*"'
```

Later (account exists):

```
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=<WEB_API_KEY>" -H
"Content-Type: application/json" -d '{"email":"test1@example.com","password":"Test-pass-
123","returnSecureToken":true}' | grep -o '"idToken": *"[^"]*"'
```

```
export TOKEN=<the idToken value>
```

Two helpers used throughout: `start` begins an asynchronous search and prints the job id; `poll` prints the job every 2 s until it is finished.

```
start() { curl -s -X POST $BASE/api/v1/searches -H "Authorization: Bearer $1" -H 'Content-Type: application/json' -d
"{\"title\":\"$2\",\"country\":\"$3\"}" | grep -o 'srch_[a-z2-7]*' | head -1; }
poll() { for i in $(seq 1 80); do OUT=$(curl -s $BASE/api/v1/searches/$1 -H "Authorization: Bearer $2"); echo "$(date
+%T) $(echo "$OUT" | grep -o '"status":"[a-z]*","request\|"markets":{[^}]*}' | tr '\n' ' ')"; echo "$OUT" | grep -q
'"status":"running","request' || break; sleep 2; done; }
```

20.3 Everyday API calls from your PC

Purpose	Command
Health	`curl $BASE/api/v1/health`
Countries	`curl $BASE/api/v1/countries`
Waiting search	`curl -s -X POST $BASE/api/v1/search -H "Authorization: Bearer $KEY" -H 'Content-Type: application/json' -d '{"title":"usb c hub","country":"jor"}'`
...with headers (cache, quota)	add `-i` to `curl`
Asynchronous search + follow it	`ID=$(start $KEY "wireless mouse" usa); poll $ID $KEY`
As an app user	`ID=$(start $TOKEN "wireless mouse" usa); poll $ID $TOKEN`
Read a job's shops in detail	`curl -s $BASE/api/v1/searches/$ID -H "Authorization: Bearer $KEY" | grep -o '"[a-z]*":{"status":"[a-z_]*"[^}]*}'`

Purpose	Command
Prices' currencies	`curl -s $BASE/api/v1/searches/$ID -H "Authorization: Bearer $KEY" | grep -o '"currency":"[^"]*"' | sort | uniq -c`
Pull to refresh	add `"refresh":true` to the JSON body
Developer docs	open `$BASE/docs/` in a browser

Webhooks (a webhook.site URL in `$HOOK`; the API client needs a webhook secret, part 12):

```
curl -s -X POST $BASE/api/v1/searches -H "Authorization: Bearer $KEY" -H 'Content-Type: application/json' -d
'{"title":"usb c cable","country":"jor","webhook_url":"'$HOOK'","webhook_events":["first_results","completed"]}'
```

MCP — start a search (answers within ~15 s):

```
curl -s $BASE/mcp -H "Authorization: Bearer $KEY" -H 'Content-Type: application/json' -H 'Accept: application/json,
text/event-stream' -H 'MCP-Protocol-Version: 2025-11-25' -d '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":
{"name":"search_products","arguments":{"title":"usb desk fan","country":"jor"}}}'
```

MCP — the rest of that search (`JOB` = the `job_id` from the answer):

```
curl -s $BASE/mcp -H "Authorization: Bearer $KEY" -H 'Content-Type: application/json' -H 'Accept: application/json,
text/event-stream' -H 'MCP-Protocol-Version: 2025-11-25' -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":
{"name":"get_search_results","arguments":{"job_id":"'$JOB'"}}}'
```

20.4 VM: log in, admin panel, deploy

```
ssh -i ~/.ssh/owis_oracle.key ubuntu@<public IP>
```

Admin panel (keep this window open, then open `http://localhost:3012/admin/`):

```
ssh -i ~/.ssh/owis_oracle.key -L 3012:127.0.0.1:3002 ubuntu@<public IP>
```

On the VM:

```
cd /opt/ofd/repo/Owis_Find_Deal_Engine/server
```

```
deploy/deploy.sh                # newest image of main
```

```
deploy/deploy.sh <commit>       # a given commit; an older one = roll back
```

```
cat /opt/ofd/deploys.log        # what was deployed when
```

20.5 VM: look at the running service

Purpose	Command
Containers and health	`docker ps --format 'table {{.Names}}\t{{.Status}}'`
Server log, live	`docker logs -f owis_find_deal_engine-server-1` (Ctrl+C to stop)
Last 30 min, filtered	`docker logs --since 30m owis_find_deal_engine-server-1 2>&1 | grep -iE "error|warn"`
Start-up facts	`docker logs owis_find_deal_engine-server-1 2>&1 | grep -E '"server started"|"search queue"|shop rules|apify scraper enabled'`

Purpose	Command
Restarts and resume	`docker logs --since 1h owis_find_deal_engine-server-1 2>&1 | grep -E 'left for the next|queued again|jobs resumed'`
Tunnel	`docker logs --tail 20 owis_find_deal_engine-cloudflared-1`
Health from inside	`curl -s http://127.0.0.1:3002/api/v1/health`
Restart the server only	`docker restart owis_find_deal_engine-server-1`
Resource use	`docker stats --no-stream`
Disk	`df -h /` and `docker system df`
Redis (read-only look)	`docker exec owis_find_deal_engine-redis-1 redis-cli --scan --pattern 'owis:q:*' | head`
Running jobs	`docker exec owis_find_deal_engine-redis-1 redis-cli smembers owis:jobs:running`
Postgres shell	`docker exec -it owis_find_deal_engine-postgres-1 psql -U ofd -d ofd`
Settings CLI	`docker exec owis_find_deal_engine-server-1 /owis_find_deal_engine settings list` (more in 16.4)

(Inside the containers `REDISCLI_AUTH` is set, so `redis-cli` needs no password.)

20.6 Local development on your PC (optional)

Full stack in Docker (needs Docker Desktop; a `.env` made from `.env.example`):

```
cp .env.example .env            # then fill in AUTH_*, passwords, keys
docker compose up -d --build
docker compose ps               # all healthy after ~30 s
curl localhost:3002/api/v1/health
docker compose logs -f server
docker compose down             # stop (add -v to also delete the data)
```

Server with Go on your PC, databases in Docker:

```
docker compose -f docker-compose.yml -f compose.dev.yml up -d searxng redis postgres
set -a; . ./.env; set +a
go run ./cmd/owis_find_deal_engine
```

Admin web app with live reload: `cd web/admin && npm ci && npm run dev` (→ `http://localhost:5173/admin/`).

20.7 Git on your PC

Purpose	Command
Latest `main`	`git checkout main && git pull`
See a work branch	`git fetch origin <branch> && git checkout <branch>`
What changed	`git log --oneline -10`, `git diff main...<branch> --stat`
Which commit runs in production	on the VM: `docker inspect owis_find_deal_engine-server-1 --format '{{ .Config.Image }}'`

Pull requests are opened and merged on github.com (repository → *Pull requests*). After merging, wait for the CI run on `main` (Actions tab, "server") to finish — it publishes the image — then run `deploy/deploy.sh`.

21. Testing

21.1 Automatic tests (every pull request)

```
cd Owis_Find_Deal_Engine/server
gofmt -l . && go vet ./... && go test -race ./...
go run golang.org/x/vuln/cmd/govulncheck@latest ./...
cd web/admin && npm ci && npm run build && npm audit --omit=dev
```

Store tests need an empty throwaway Postgres: `TEST_DATABASE_URL=postgres://user:pass@127.0.0.1:5432/ofd_test?sslmode=disable go test -race ./internal/store`. CI runs all of this on GitHub (part 4).

Every package has unit tests; notable ones:

Area	Tests
Queue	join, cap per class, queue wait, searches before refreshes, promotion, restart re-run once, crash recovery, too old — on memory **and** Redis (miniredis)
Cache	hit, shared between countries, only missing shops live, last_known and its 7-day end, stale refresh, forced refresh, errors not stored, progress updates, slow shop missing the wait then stored, currency
Jobs	progress saves, running list, stop and resume, cancel cause
API	job filled per shop (app user), webhooks in order with partial body, events validation, MCP wait + `get_search_results`, resume with a lost job
Metering	counted once, free when cached, refunds, stopped-for-resume stays counted

21.2 End-to-end test (Claude's sandbox)

Before each release Claude runs the **real server binary** with Postgres, Redis and fake Firebase / SerpApi / Apify servers, and checks the main scenarios (results per shop, joining, cap, webhooks, MCP, restart during a search). The harness is kept outside the repository.

21.3 Owner tests in production

Each step ends with a test list in `STATUS.md` that you run with the commands of part 20. Step 14b parts 3 + 4 (2026-11-11), all passed:

#	Test	What to see
0	start log	`search queue` apify 4 / serpapi 10 / max_wait 1m0s; aliexpress `apify_run_timeout 1m25s`
1	app user, new title, `poll`	fast shops first, slow ones `pending`, then `done`; prices `USD`
2	same title again	`202` already `done`, `cached:true`
3	webhooks with webhook.site	`search.first_results` then `search.completed`; `["x"]` → `400`
4	MCP `search_products`	`done` or `running` + `pending` + `job_id`; `get_search_results` for the rest
5	restart 2 s after starting (on the VM)	job ends `done`; log: left / queued again / resumed
6	same new title from two callers	one Apify run per shop
9	waiting search	full answer

Optional (when AliExpress is slow again): 7 Apify stop margin, 8 memory.

22. Troubleshooting

Symptom	Cause	Fix
`400 webhook_not_configured`	the API client has no webhook secret	admin panel → API clients → Webhook secret; wait up to a minute
`401 unauthorized` with a login token	token older than 1 hour, wrong project, or a sign-in method not allowed (anonymous)	get a new token; check `AUTH_ALLOWED_SIGN_IN`
`401` with an API key	revoked / expired key, or typo	admin panel → API clients
`403 forbidden` / `country_not_allowed`	key scope, IP allow-list or countries	change the client
`402` / `429 quota_exceeded`	daily allowance used up	wait for UTC midnight, or raise the plan / client quota
A test search comes back `done` and `cached:true` at once	that title was searched in the last 48 h (also from another country)	use a new title
`/admin` gives `403`	blocked on purpose on the public address	use the SSH tunnel (`localhost:3012/admin/`)
Git Bash: "no such file" with `/owis_find_deal_engine`	Git Bash converts `/paths`	`export MSYS_NO_PATHCONV=1` or write `//owis_find_deal_engine`
A shop always `error`	source failing (key, actor, scraper broken)	admin → Test search for that shop; server log `shop search failed` ; Apify / SerpApi console
A shop `last_known` often	slower than its time limit	raise `SHOP_<SHOP>_TIMEOUT` (keep the waiting search under 125 s), try `APIFY_<SHOP>_MEMORY_MB`
Shops stay `pending` long	queue full (many paid runs at once)	`QUEUE_APIFY_MAX_RUNS` , look at the log
`524` from Cloudflare	an answer took over 125 s (waiting search)	use the asynchronous search; lower shop limits
`deploy.sh` : "no image for ..."	CI did not build that commit (no server change or CI failed)	GitHub → Actions; deploy without argument
`deploy.sh` : server unhealthy	bad settings / start error	read the printed log; roll back with the printed command
Tunnel not connected	wrong `TUNNEL_TOKEN` / Cloudflare	`docker logs owis_find_deal_engine-cloudflared-1`
Settings change has no effect	applied within 10 s; or it is a bootstrap key	check `settings applied` in the log; bootstrap keys need `.env` + redeploy

23. Current status and remaining work

Done (phases 1–4 and step 14b parts 1–4): search service with SerpApi / Apify / web sources, per-shop cache with last known results, admin panel and admin API, API clients and keys, asynchronous search with signed webhooks, MCP for agents, deployment on Oracle + Cloudflare, rules per shop, queue with caps and joining, jobs filled per shop, restart recovery.

Remaining (details in `STATUS.md` → "Remaining work"):

Area	Items
Search rules (14b)	part 5 new sources (eBay Browse API — Production keyset ready; SHEIN source with D8; AliExpress affiliate API); part 6 empty-shop guard + shop-rules page; part 7 guide PDF; Apify stop-margin and memory tests when AliExpress is slow
Deployment (14)	step 5 admin two-factor login + verified email; step 6 Cloudflare Access on `/admin`; nightly Postgres backups to Oracle Object Storage; monitoring and alerts; a real-agent MCP test; staging before real users
Owner	Oracle Pay-As-You-Go upgrade before the trial ends (~1 December 2026); D8 paid Apify / SerpApi plans; Cloudflare account (registrant email, two-factor, `dealcoon.com` auto-renew off); payment provider
Phase 5 — users	`users` + `/me` + per-IP free limit; plans + payments; saved cart + price tracker; clicks + purchase reports; notifications (inbox, FCM push)
Later	search by photo (Google Lens), scraping API instead of Apify, direct shop links resolved on tap

24. Glossary

Term	Meaning
shop / market	an online store (Amazon, AliExpress, ...); `market` is its id in the code
source	who searches a shop: SerpApi engine, Apify scraper, web chain
live search	a search that asks a source now (costs money, counts against the allowance)
cached answer	served from Redis without asking a source (free)
last_known	a shop's latest results (≤ 7 days) shown when its live search fails
job	one asynchronous search (`srch_…`), kept in Redis
task	one shop's live search in the queue; joined by everyone searching the same shop + title
class	a group of sources sharing a cap: `apify` , `serpapi` , `web`
fast / slow group	per shop; `first_results` is sent when the fast shops are done
allowance / quota	daily number of live searches (per plan, per API client)
JWKS	the public keys that verify Firebase login tokens
MCP	Model Context Protocol — how AI agents call tools
tunnel	Cloudflare Tunnel: carries public traffic to the VM over an outbound connection
bootstrap / runtime setting	`.env` -only setting / setting stored in Postgres and changed live
settings version	counter that grows with every settings change; the server rebuilds when it changes
cache generation	number in every cache key; raising it drops every cached answer

Generated from the repository state of 11 November 2026. Rebuild the PDF with `python3 docs/build_full_reference_pdf.py` (see that file). Diagrams: `python3 docs/reference/make_diagrams.py` .